Data Structures and Algorithms

Lecture 12

Graphs and Graph Algorithms

Dept. of ICT, Faculty of Technology- UoR, 2026

Learning Objectives

- To learn what a graph is and how it is used.
- To implement the graph abstract data type using multiple internal representations.
- To see how graphs can be used to solve a wide variety of problems.

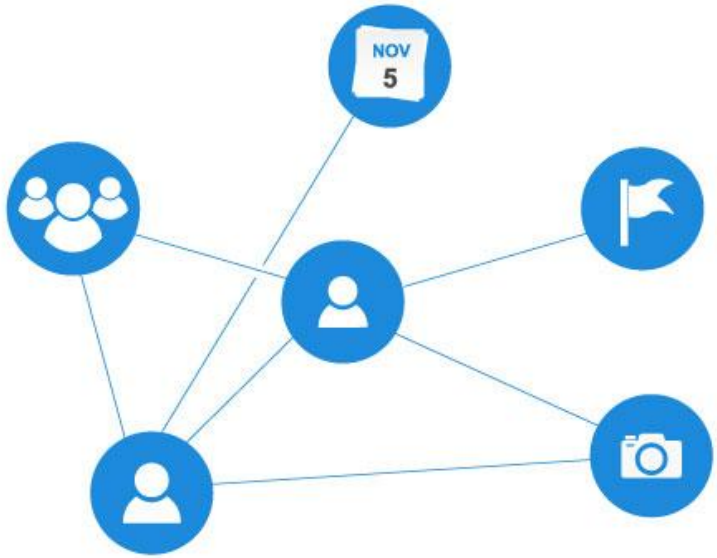
What is graph?

- A graph data structure is a collection of nodes that have data and are connected to other nodes.
- Graphs are a more general structure than the trees.
- Tree is a special kind of graph.
- Graphs can be used to represent many interesting things
  - Our world, roads,
  - Airline flights from city to city,
  - How the Internet is connected,
  - Or even the sequence of classes you must take to complete a ICT degree

Graph -examples

- On Facebook, everything is a node. That includes User, Photo, Album, Event, Group, Page, Comment, Story, Video, Link, Note...anything that has data is a node.

- Every relationship is an edge from one node to another. Whether you post a photo, join a group, like a page etc., a new edge is created for that relationship.

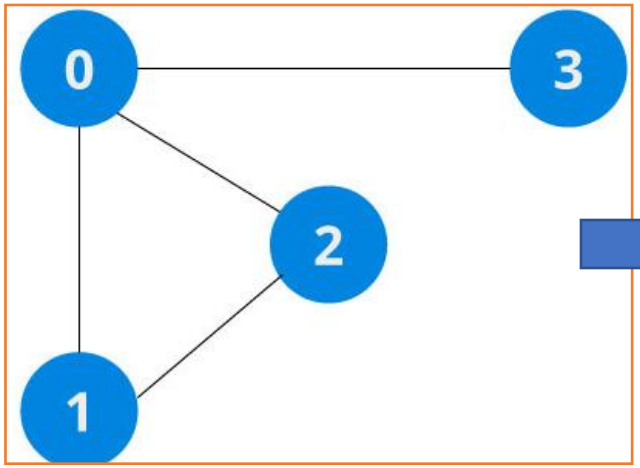


Graph- Definition

A graph is a data structure (V,E) that consists of

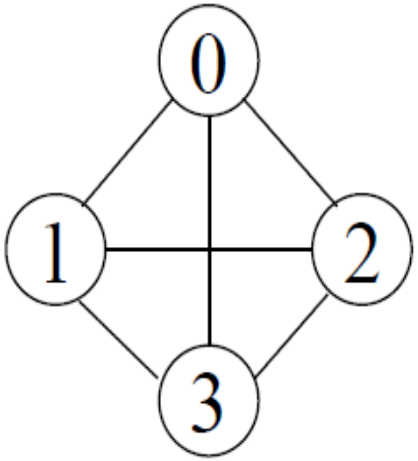
- **A collection of vertices V** (A vertex is also called a “node”)
- **A collection of edges E** (An edge is also called an “arc”)

represented as ordered pairs of vertices (u,v)

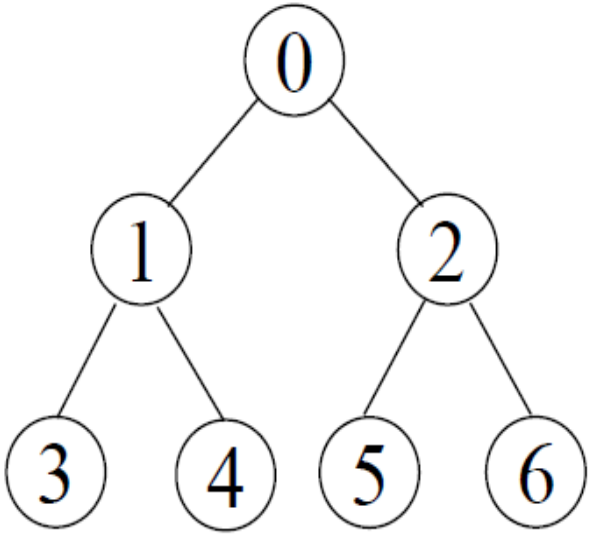


V = {0, 1, 2, 3}
E = {(0,1), (0,2), (0,3), (1,2)}
G = {V, E}

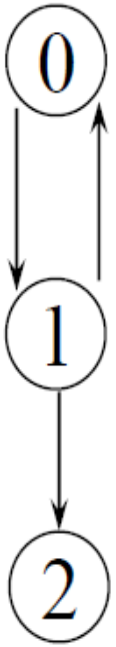
Examples for Graph



G_1

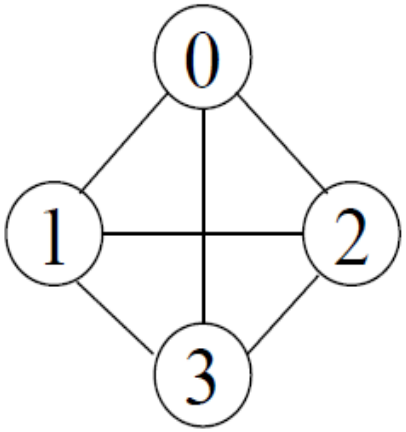


G_2

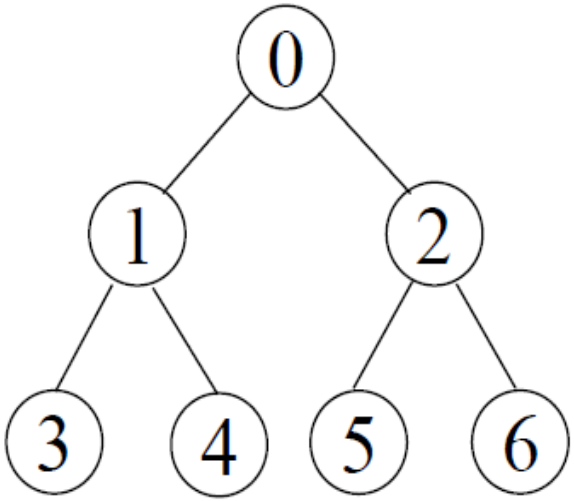


G_3

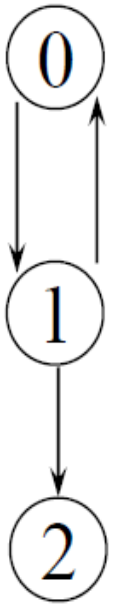
Examples for Graph



G_1



G_2



G_3

$V(G_1)=\{0,1,2,3\}$

$E(G_1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$

$V(G_2)=\{0,1,2,3,4,5,6\}$

$E(G_2)=\{(0,1),(0,2),(1,3),(1,4),(2,5),(2,6)\}$

$V(G_3)=\{0,1,2\}$

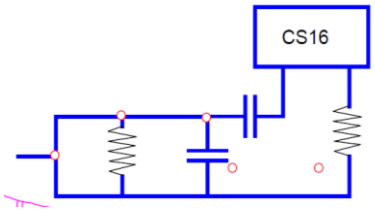
$E(G_3)=\{<0,1>,<1,0>,<1,2>\}$

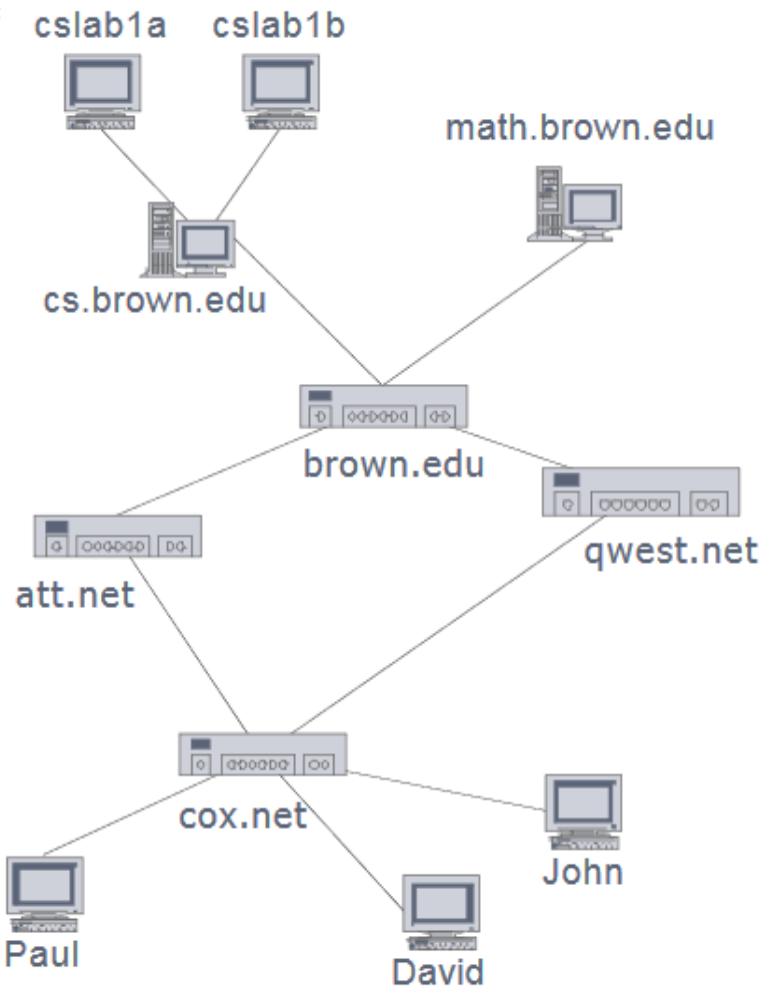
What's different? Trees and Graphs

- Tree
  - One node, the **root**, is "distinguished". Almost always the starting point for algorithms
  - Every node has exactly one **parent** (but the root has no parent)
    - parent and child hierarchy
    - **Leaf** nodes, **internal** nodes
  - One distinct **path** between two nodes
- Graph
  - No **parent/child** relationship
  - "Neighbors" or **adjacent nodes**
  - Possibly multiple paths between two nodes; also **cycles** possible

Example Applications

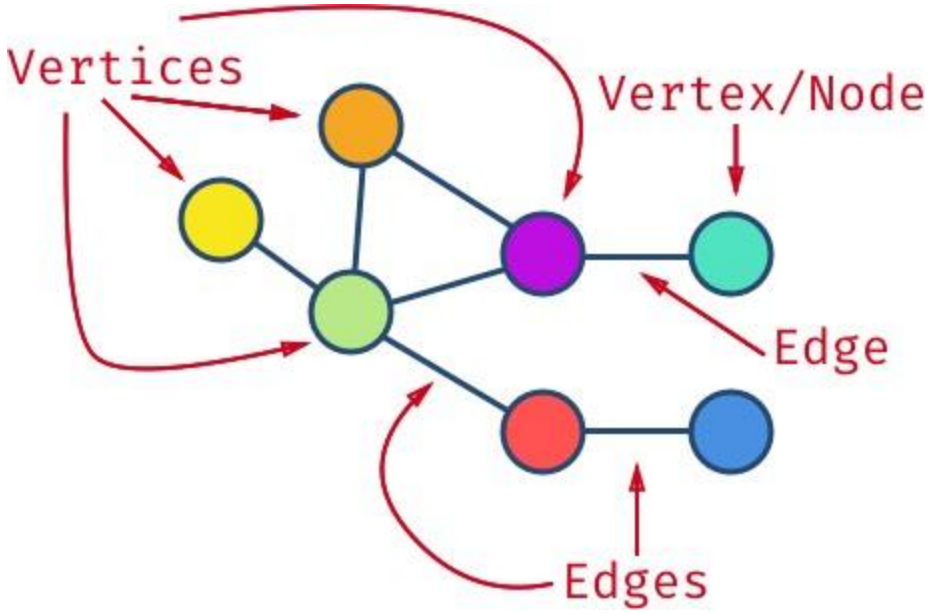
- Electronic circuits
  - Printed circuit board
  - Integrated circuit
- Transportation networks
  - Highway network
  - Flight network
- Computer networks
  - Local area network
  - Internet
  - Web
- Databases
  - Entity-relationship diagram
- Questions
  - Are two points connected?
  - What is the shortest path between them?





Graphs Basics

The connection between two nodes is called **edge**. Nodes can also be called **vertices**.



Vertex
city
computer
web page
airport

Edge
road
cable
hyperlink
flight

The **degree** : Number of edges connected to a vertex. E.g., the purple vertex has a degree of 3 while the blue one has a degree of 1.

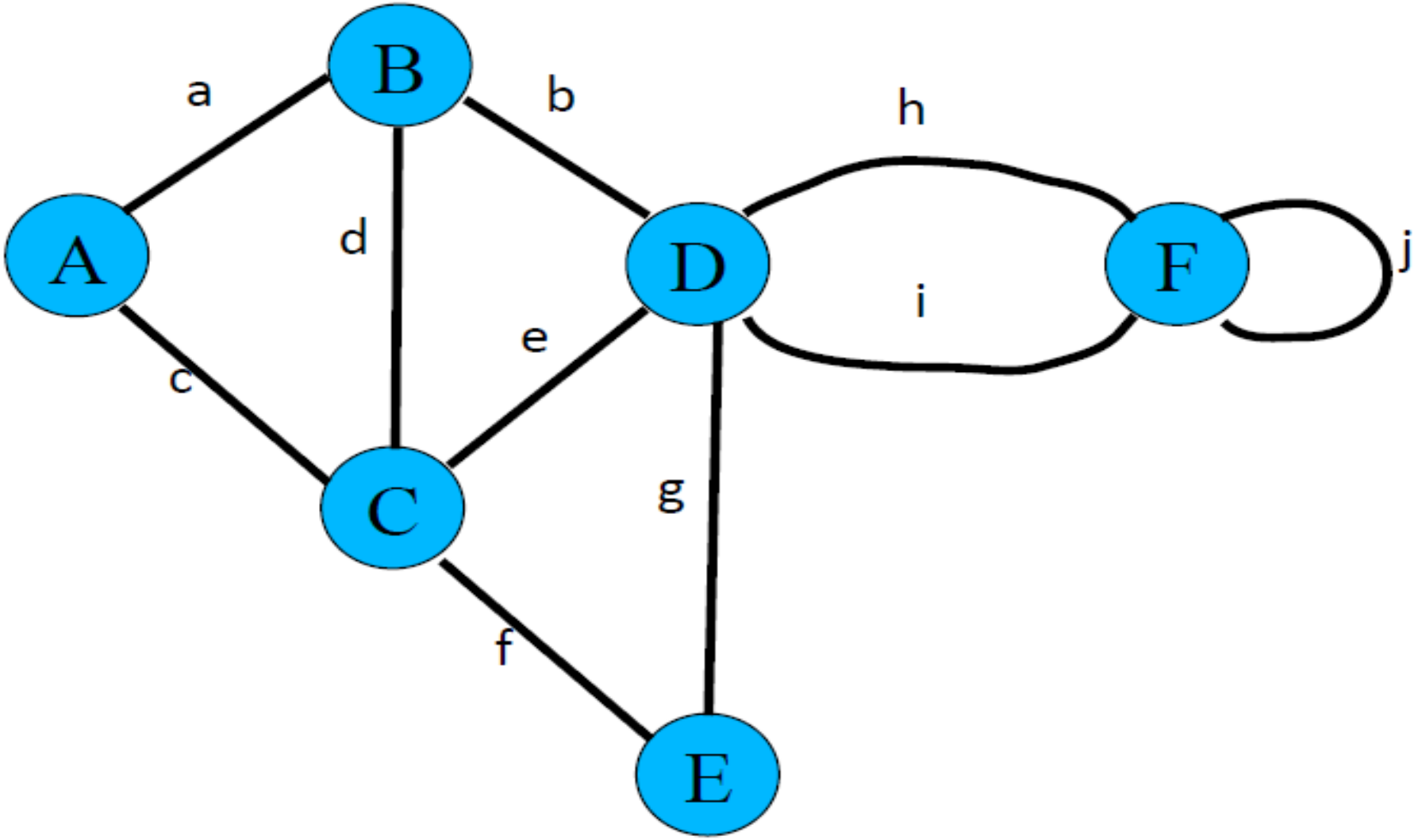
Graph Terminology

- Two vertices joined by an edge are called the **end vertices** or **endpoints** of the edge.

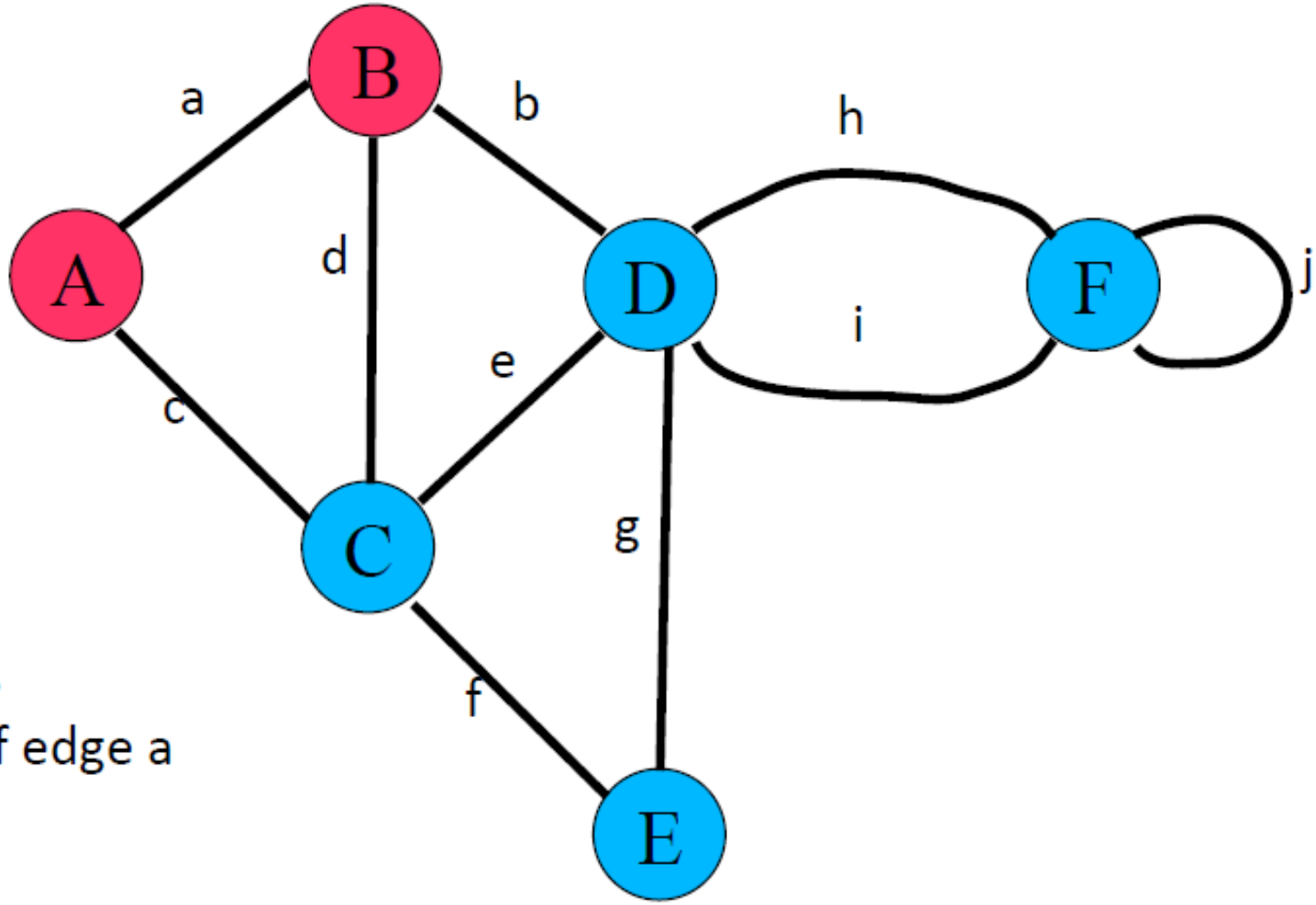
- If an edge is directed its first endpoint is called the **origin** and the other is called the **destination**.

- Two vertices are said to be **adjacent** if they are endpoints of the same edge.

Graph Terminology

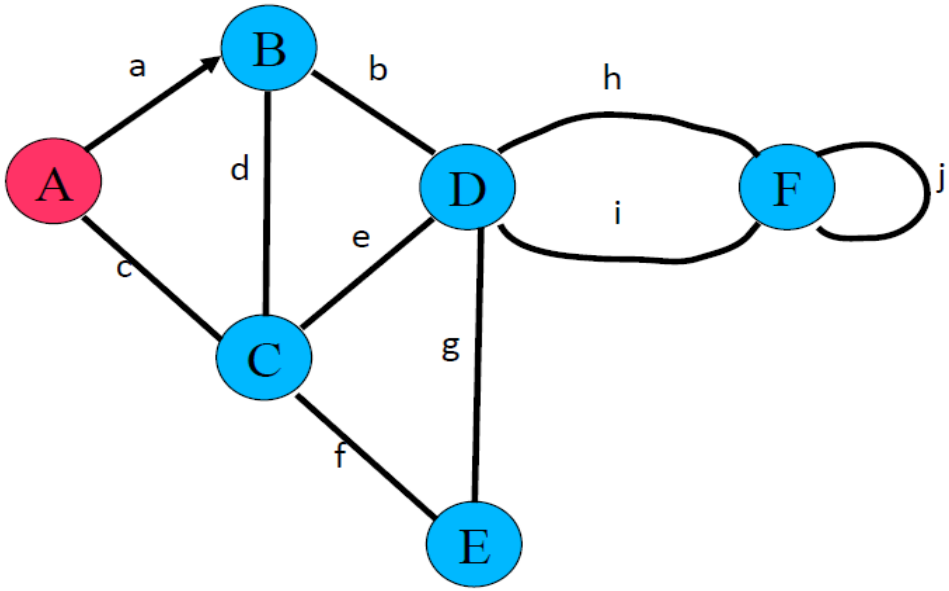


Graph Terminology

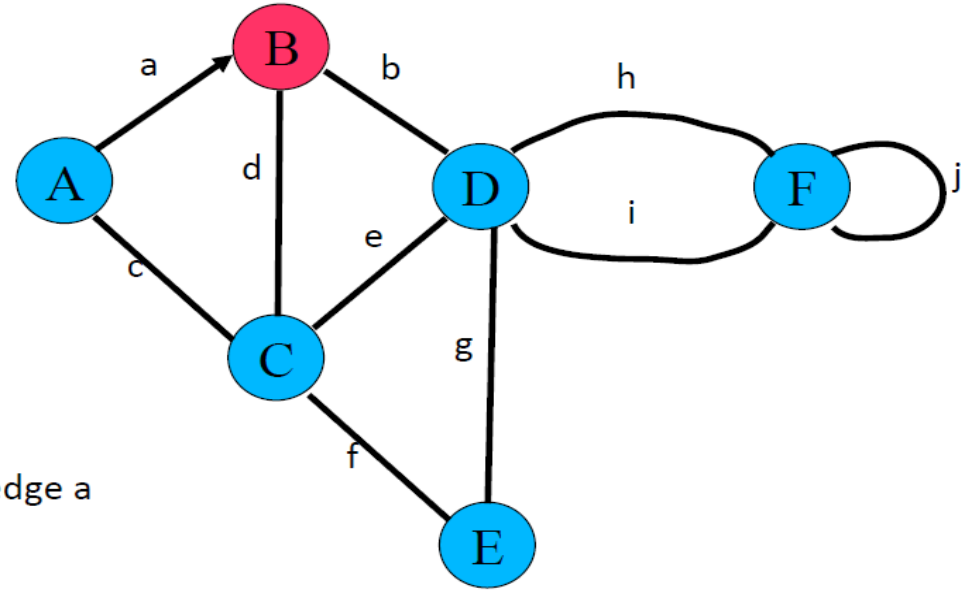


Vertices A and B
are endpoints of edge a

Graph Terminology

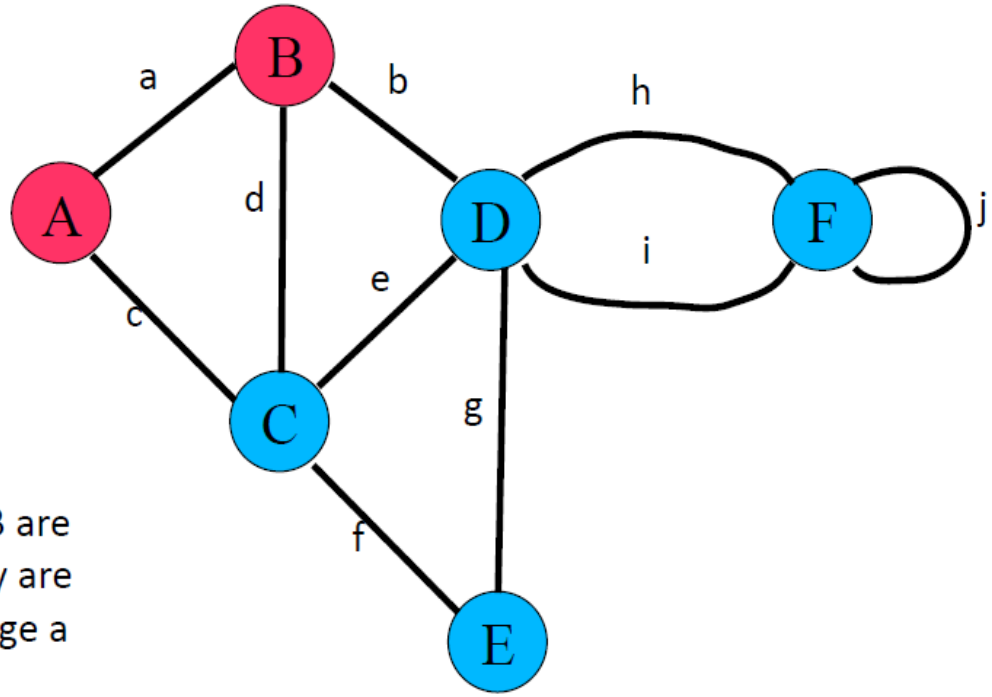


Vertex A is the origin of edge a



Vertex B is the destination of edge a

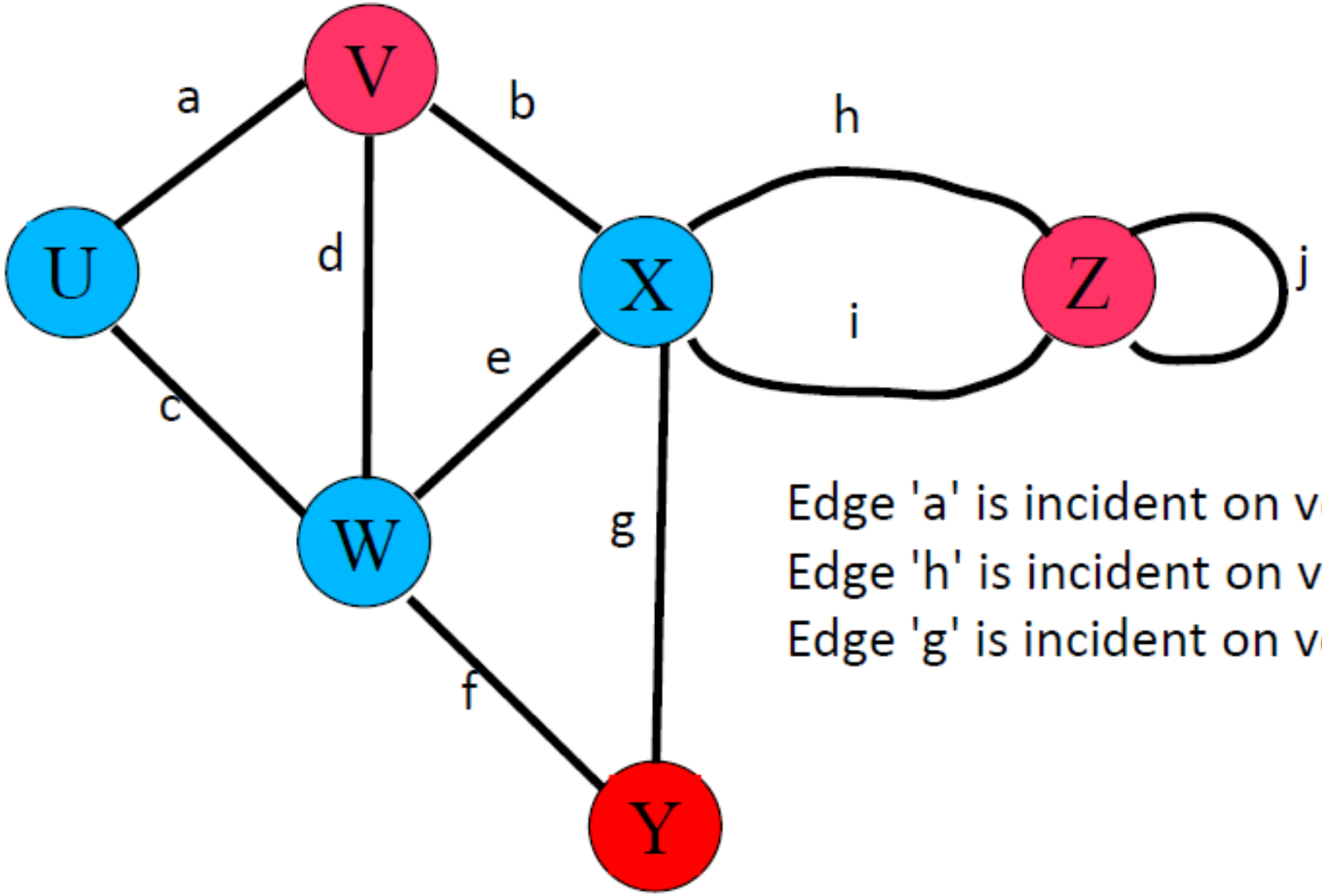
Graph Terminology



Vertices A and B are adjacent as they are endpoints of edge a

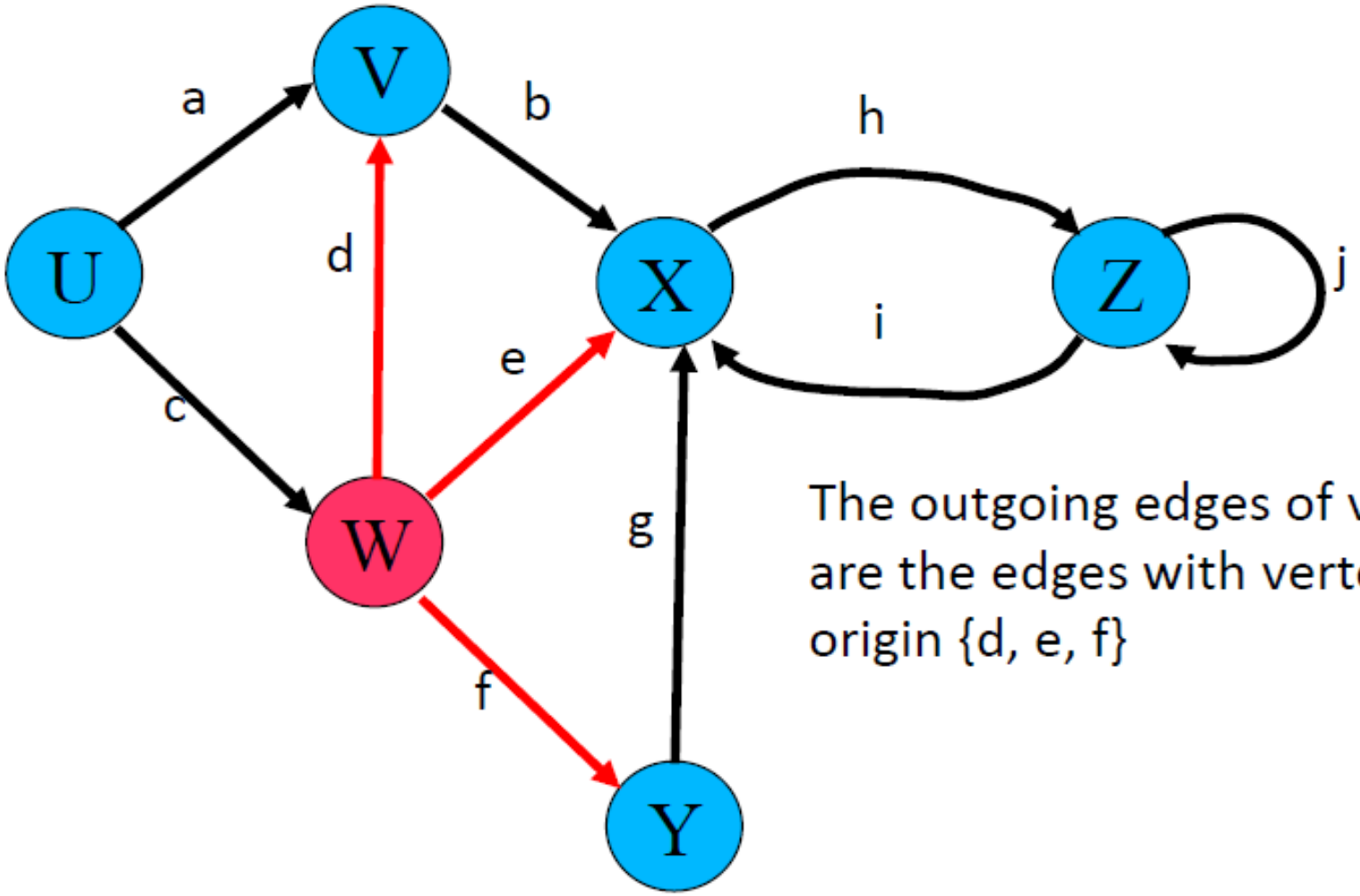
- An edge is said to be **incident** on a vertex if the vertex is one of the edges endpoints.
- The **outgoing** edges of a vertex are the directed edges whose origin is that vertex.
- The **incoming** edges of a vertex are the directed edges whose destination is that vertex.

Graph Terminology



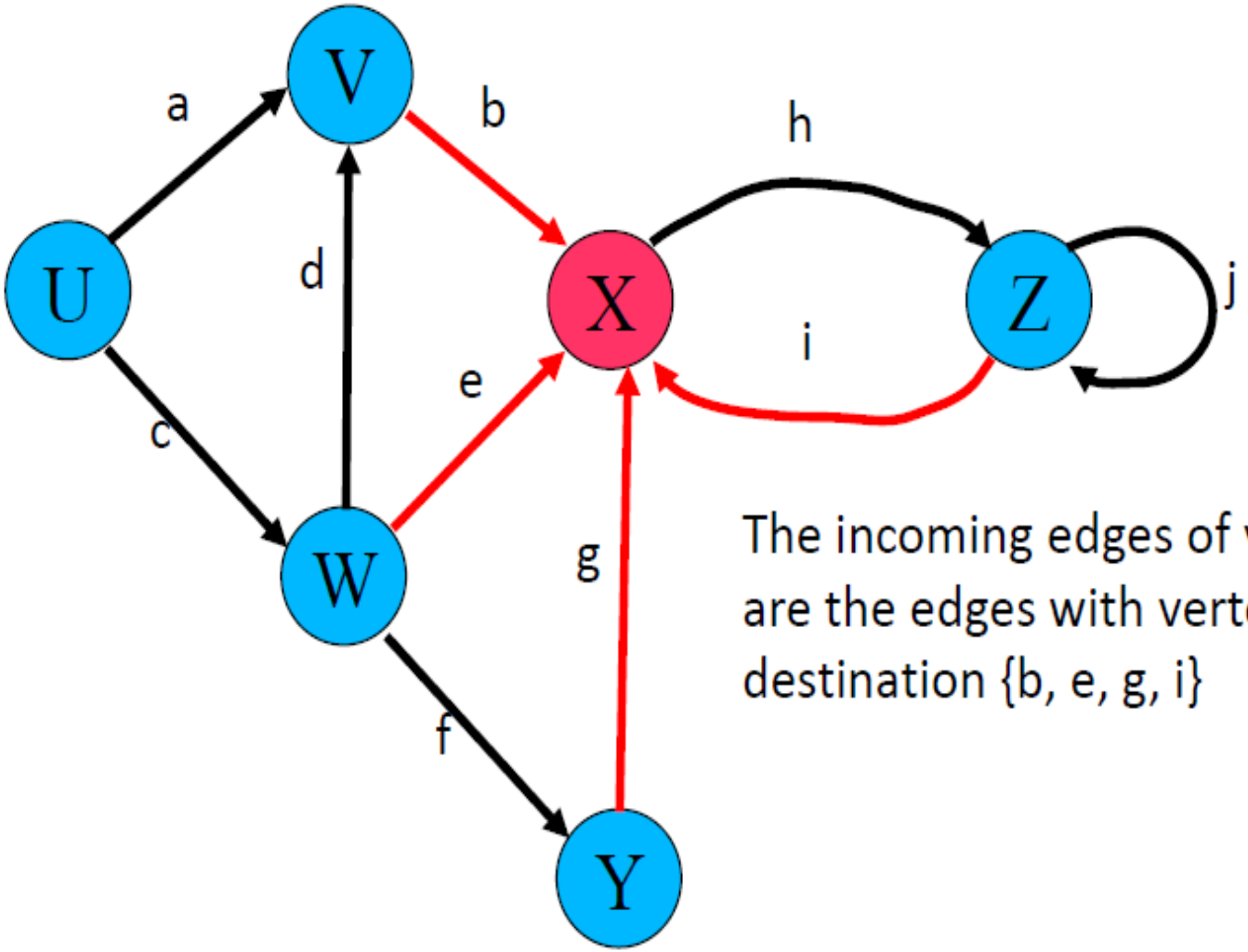
Edge 'a' is incident on vertex V
Edge 'h' is incident on vertex Z
Edge 'g' is incident on vertex Y

Graph Terminology



The outgoing edges of vertex W are the edges with vertex W as origin {d, e, f}

Graph Terminology



The incoming edges of vertex X are the edges with vertex X as destination {b, e, g, i}

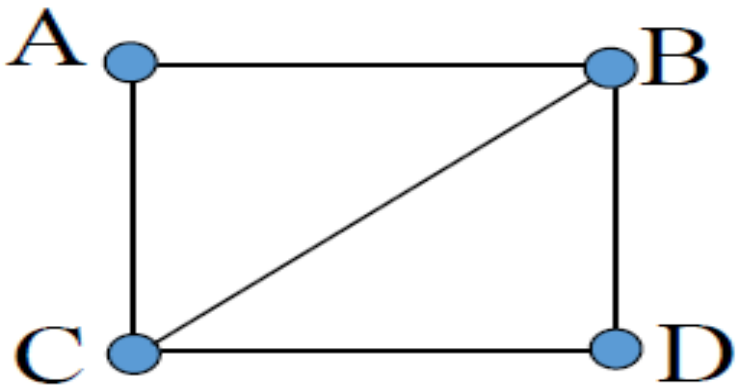
Adjacent, neighbors

- Two vertices are adjacent and are neighbors if they are the endpoints of an edge

  Example:

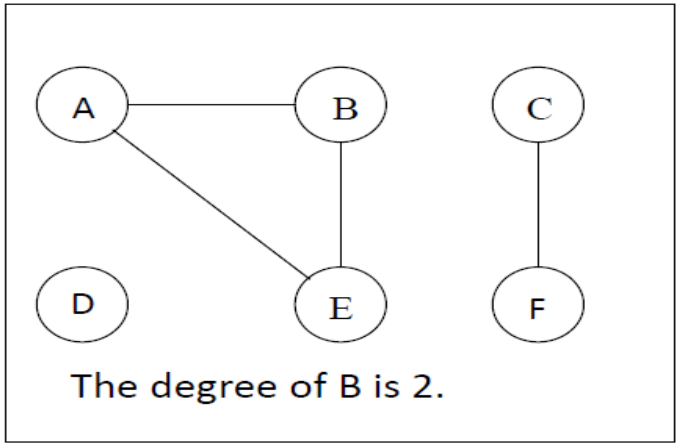
  A and B are adjacent

  A and D are not adjacent



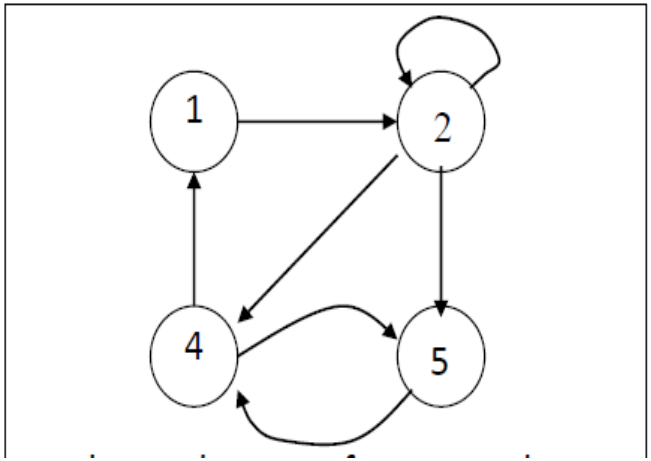
Degree

Degree: Number of edges incident on a node



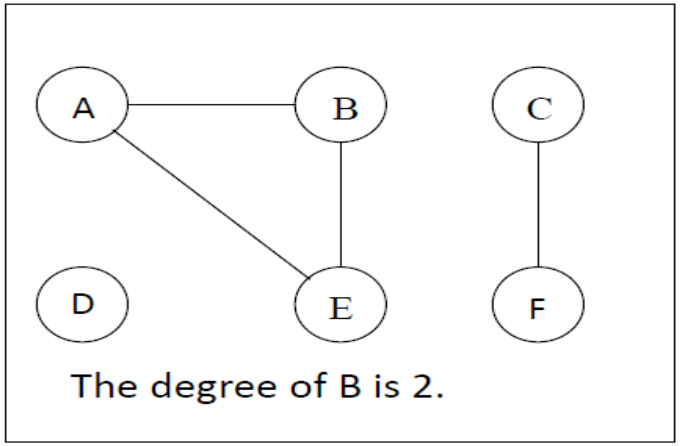
Degree (Directed Graphs)

- In degree: Number of edges entering a node
- Out degree: Number of edges leaving a node
- Degree = Indegree+ Outdegree



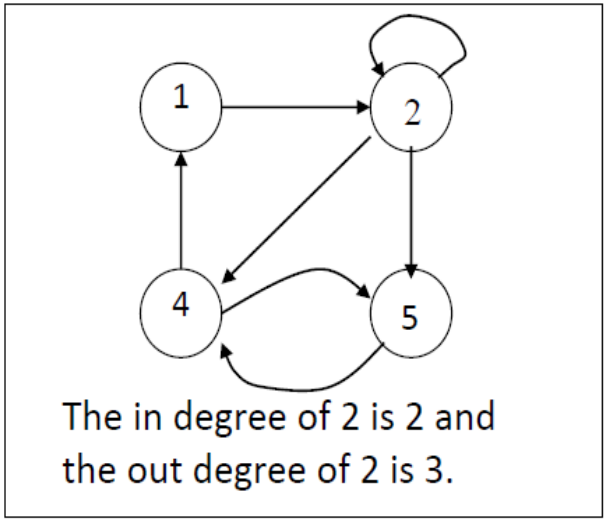
Degree

Degree: Number of edges incident on a node



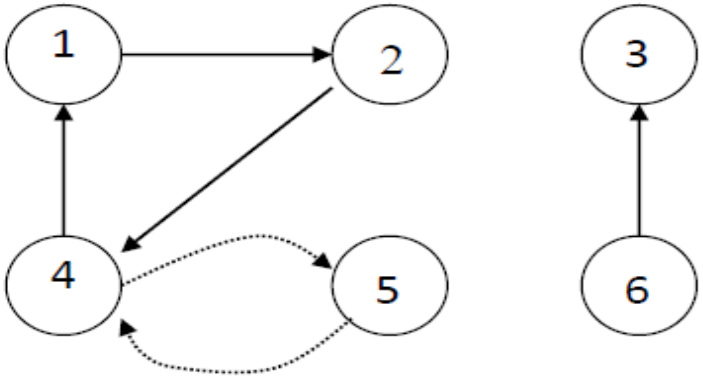
Degree (Directed Graphs)

- In degree: Number of edges entering a node
- Out degree: Number of edges leaving a node
- Degree = Indegree+ Outdegree



Path

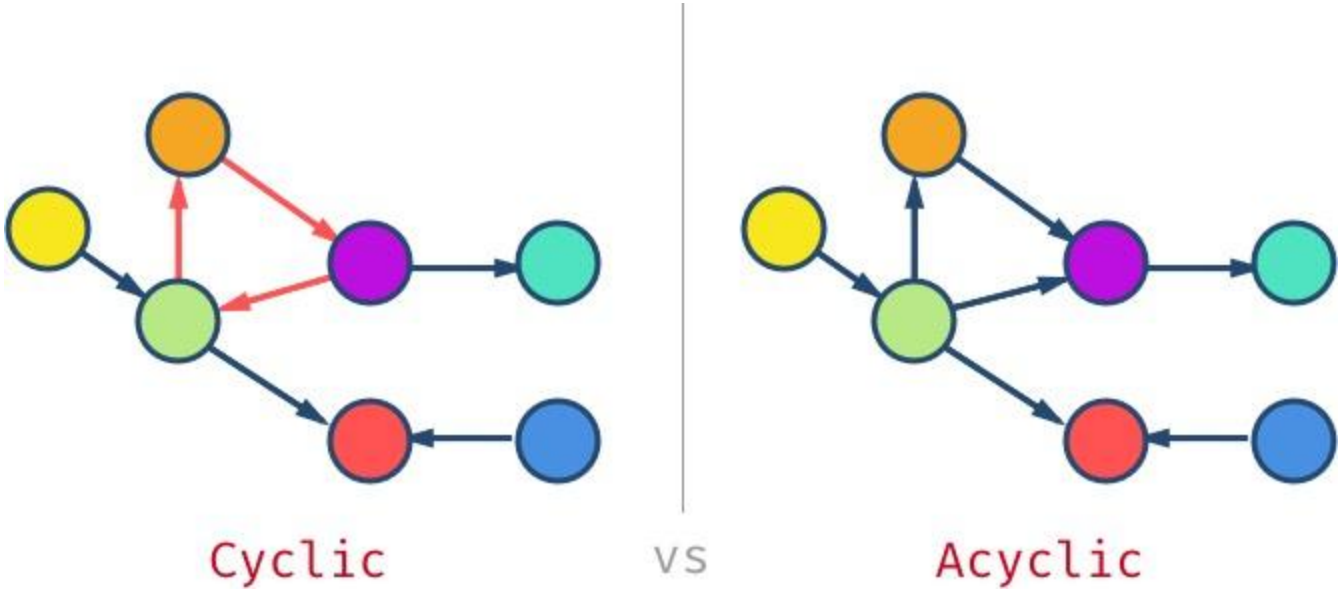
- A path is a sequence of vertices such that there is an edge from each vertex to its successor.
- A path is **simple** if each vertex is distinct.
- A **circuit** is a path in which the terminal vertex coincides with the initial vertex



Simple path: [1, 2, 4, 5]
Path: [1, 2, 4, 5, 4]
Circuit: [1, 2, 4, 5, 4, 1]

Cycles

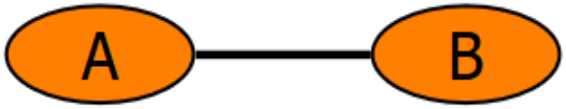
A graph can have **cycles** which means that if you traverse through the node, you could get to the same node more than once.
The graph without **cycles** is called **acyclic graph**.



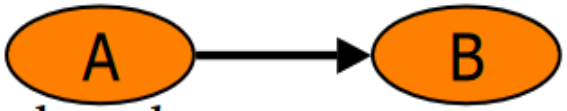
Type of graphs

Types of edges

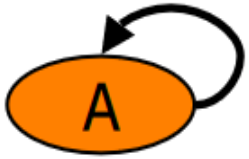
- Undirected edge {u,v}
  - Does not indicate direction
  - We can "travel" in either direction



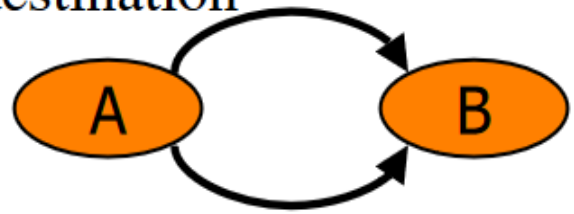
- Directed edge (u,v)
  - Has a direction
  - We can only "travel" in one direction along the edge

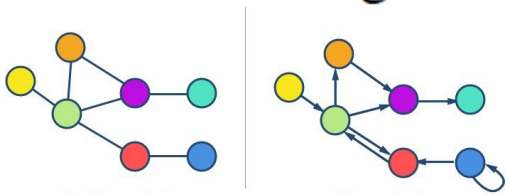


- Self-loop
  - Edge that originates and ends at the same vertex



- Parallel edges
  - Two undirected edges with same end vertices
  - Two directed edges with same origin and destination





Type of graph

Null graph, Trivial Graph

- A graph G=(V,E) where E=0 is said to be Null or Empty graph



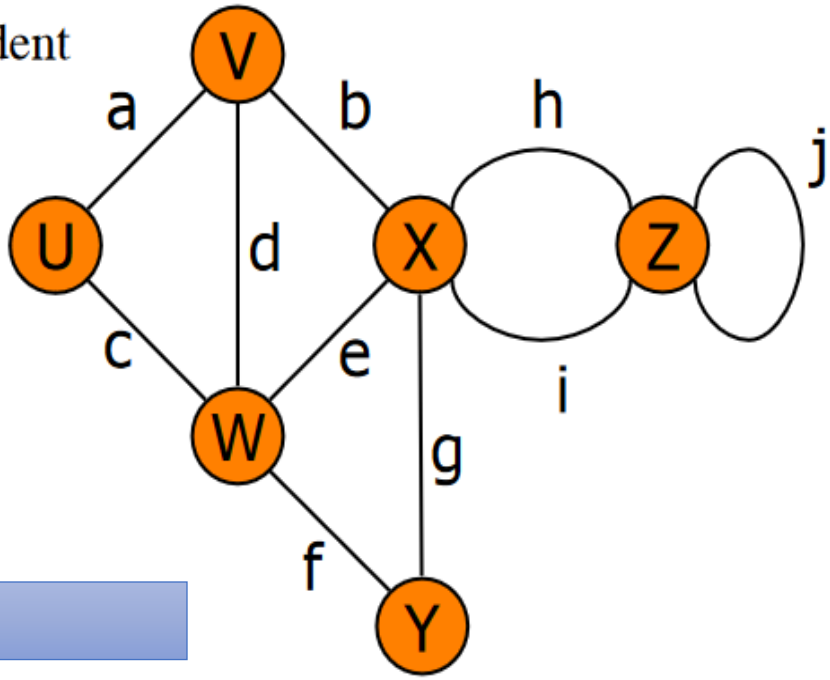


- A graph with One vertex
  and no edge is called as a trivial graph.



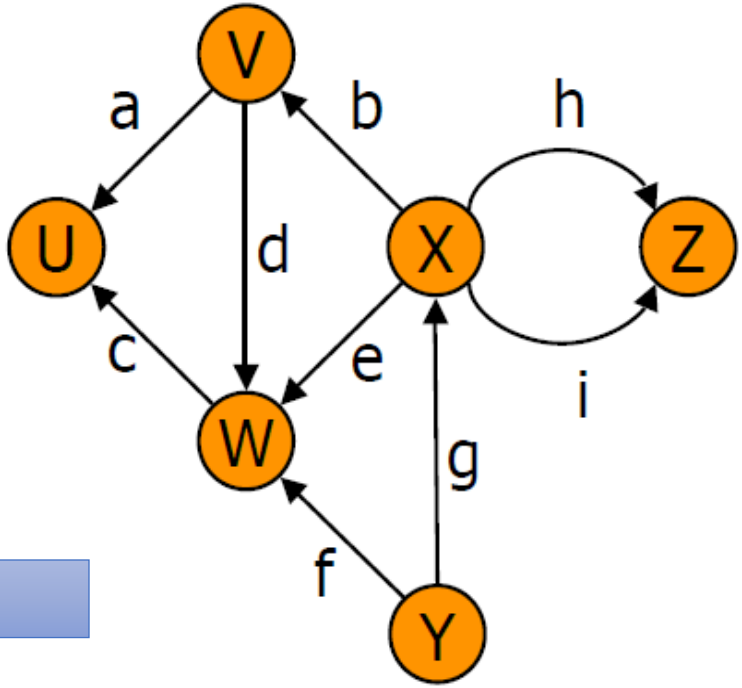
Vertices and Edges

- End vertices (or endpoints) of an edge
  - U and V are the endpoints of a
- Edges incident on a vertex - all edges that have the vertex as endpoint
  - a, d, and b are incident on V
- Adjacent vertices - endpoints of the same edge
  - U and V are adjacent
- Degree of a vertex - number of edges incident
  - X has degree 5
- Parallel edges
  - h and i are parallel edges
- Self-loop
  - j is a self-loop



Vertices - directed graphs

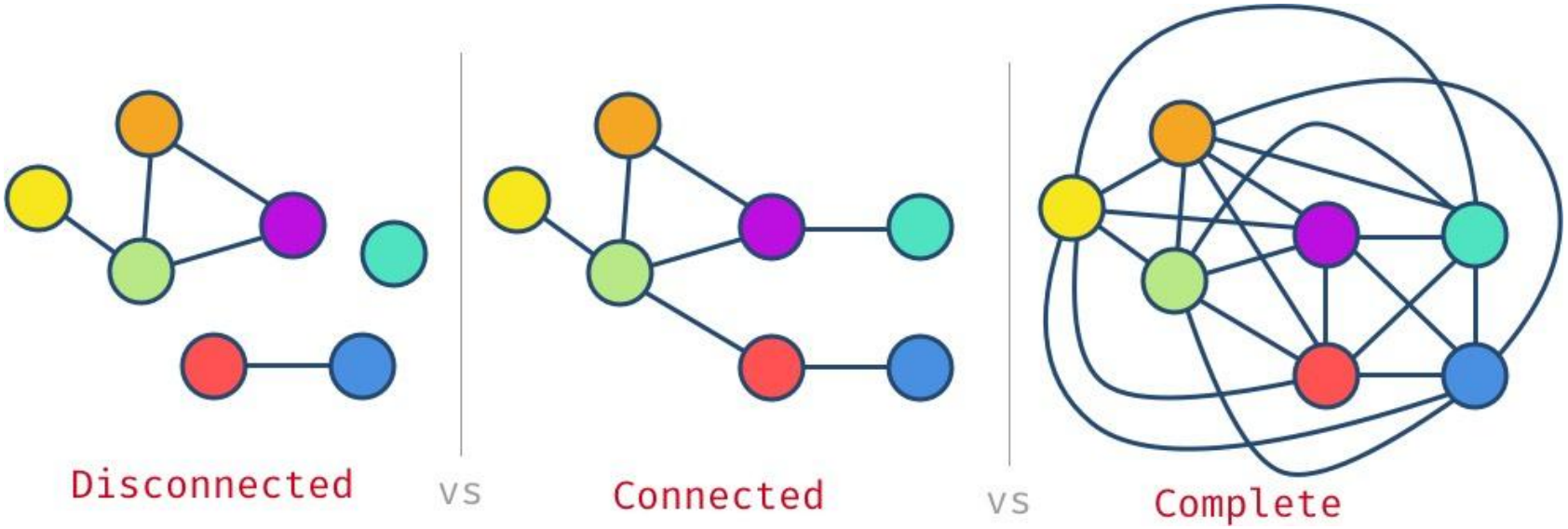
- End points
  - Origin - first endpoint
  - Destination - second endpoint
- Outgoing edges of a vertex - directed edges that originate at the vertex
  - a and d originate in V
- Incoming edges of a vertex - directed edges that terminate at the vertex
  - a and c terminate at U
- Degree of a vertex
  - Number of incoming and outgoing edges



Type of graph

- Vertices have to be connected in the graph. You might have isolated nodes or even separated subgraphs.

- If all nodes have at least one edge, then Its a **connected graph**. When all nodes are connected to all other nodes, then we have a **complete graph**.



- For a complete graph, each node has to have **#nodes - 1 edges**.

Graph Representations

Graph data structure is represented using following representations...

- Adjacency Matrix
- Incidence Matrix
- Adjacency List

Adjacency Matrix

Graph can be represented using a matrix of size total number of vertices by total number of vertices.

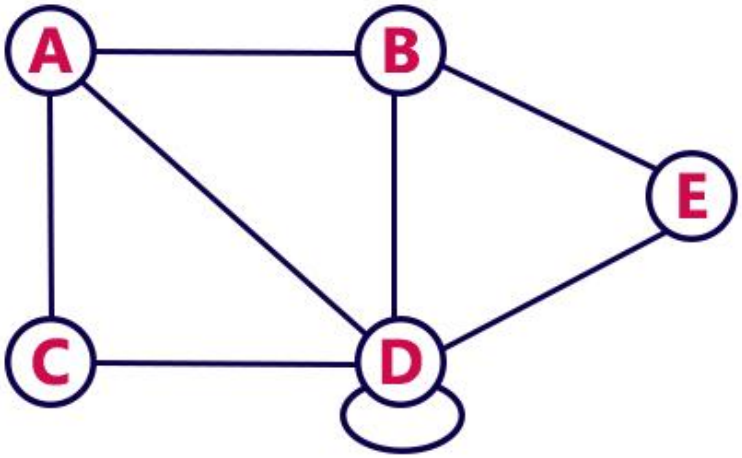
Ex:

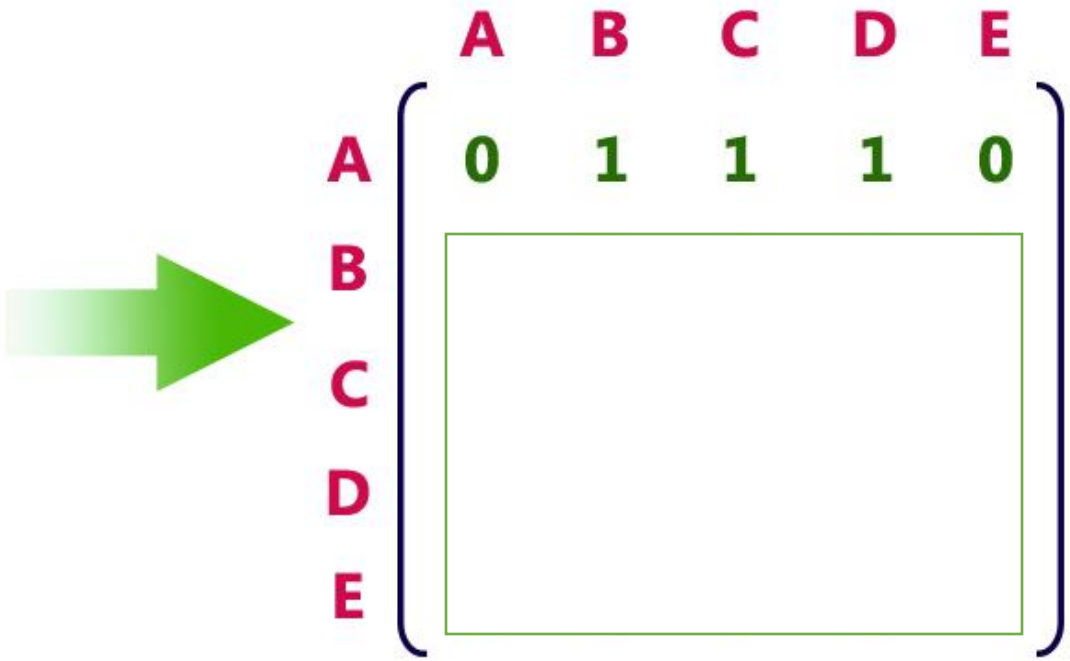
- A graph with 4 vertices can be represented using a matrix of **4X4** class.

- In this matrix, rows and columns both represents vertices and filled with either **1 or 0.**

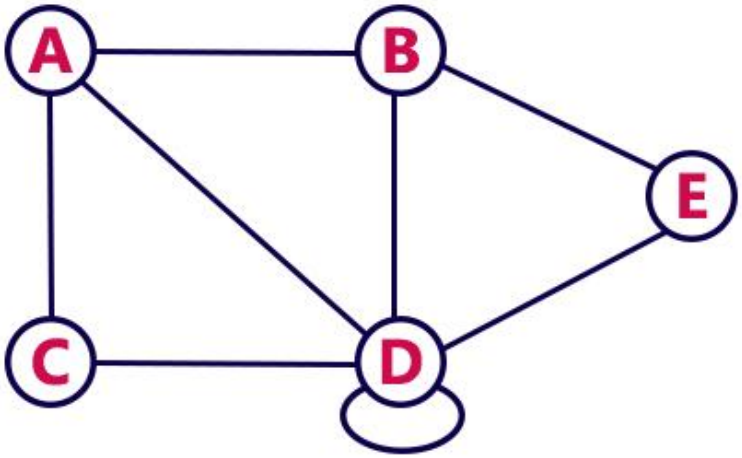
1 :represents there is an edge from row vertex to column vertex and
0 :represents there is no edge from row vertex to column vertex

Undirected graph representation...



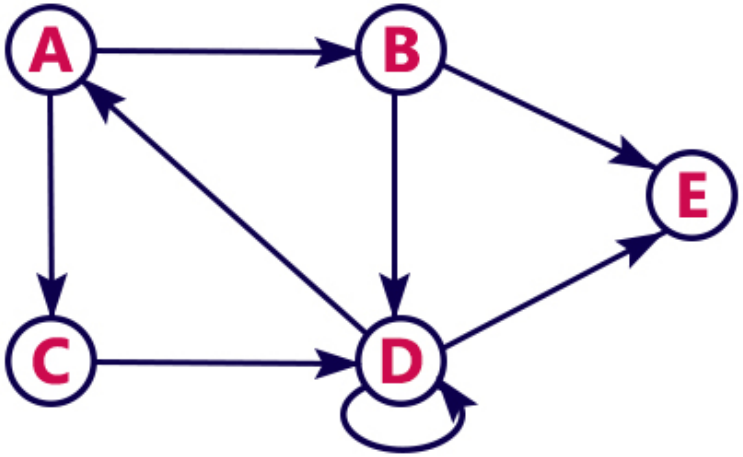


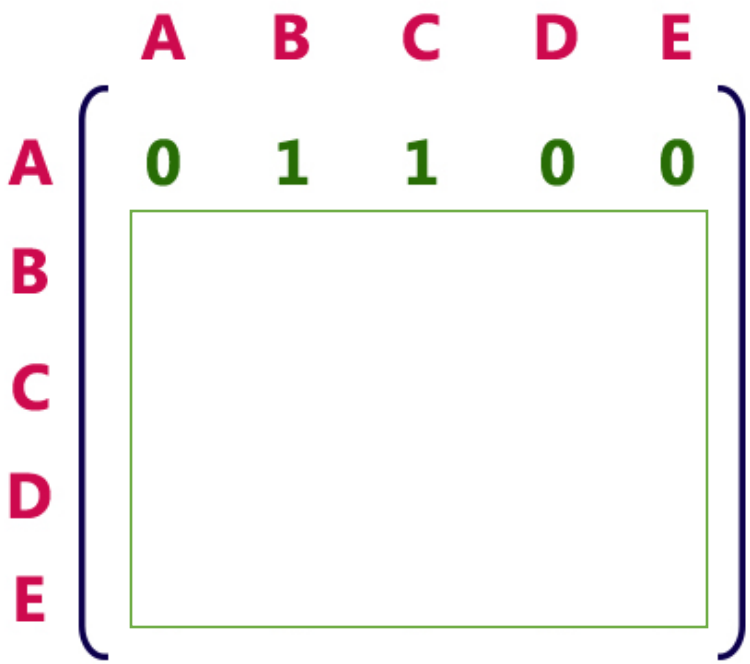
Undirected graph representation...



	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	1	1
E	0	1	0	1	0

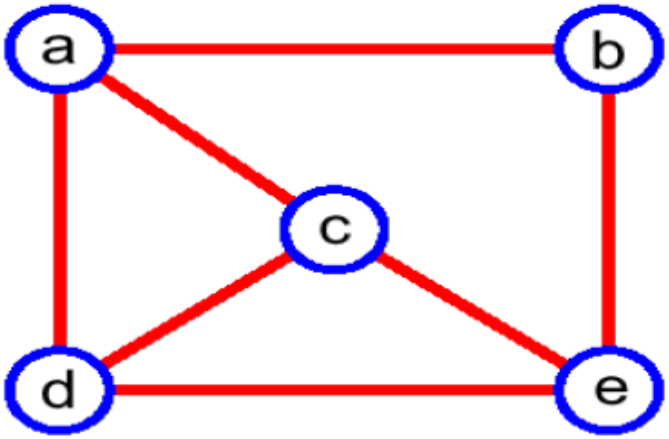
Directed graph representation...





Adjacency Matrix

- Matrix M with entries for all pairs of vertices
- M[i,j] = true – there is an edge (i,j) in the graph
- M[i,j] = false – there is no edge (i,j) in the graph



	a	b	c	d	e
a	F	T	T	T	F
b	T	F	F	F	T
c	T	F	F	T	T
d	T	F	T	F	T
e	F	T	T	T	F

Adjacency Matrix

Graph Representation

Vertex List

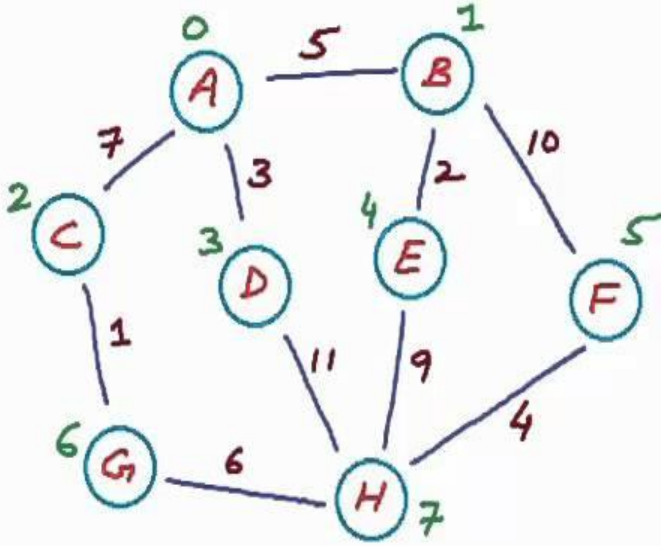
0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

Adjacency Matrix

	0	1	2	3	4	5	6	7
0	∞	5	7	3	∞	∞	∞	∞
1	5	∞	∞	∞	2	10	∞	∞
2	7	∞	∞	∞	∞	∞	1	∞
3	3	∞	∞	∞	∞	∞	∞	11
4	∞	2	∞	∞	∞	∞	∞	9
5	∞	10	∞	∞	∞	∞	∞	4
6	∞	∞	1	∞	∞	∞	∞	6
7	∞	∞	∞	6	11	9	4	∞

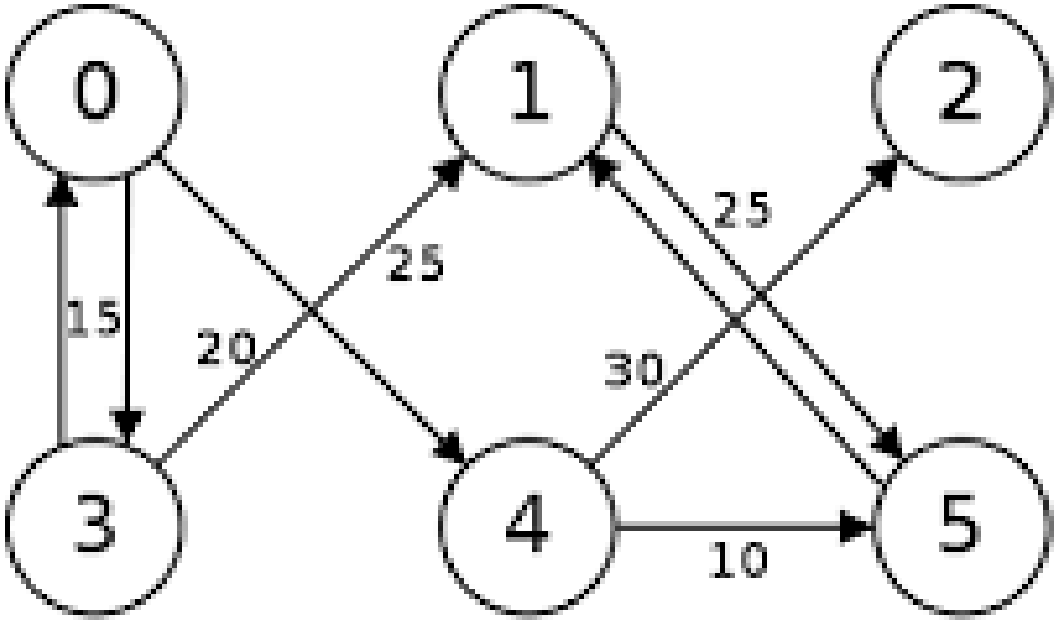
A

$|V| = v$



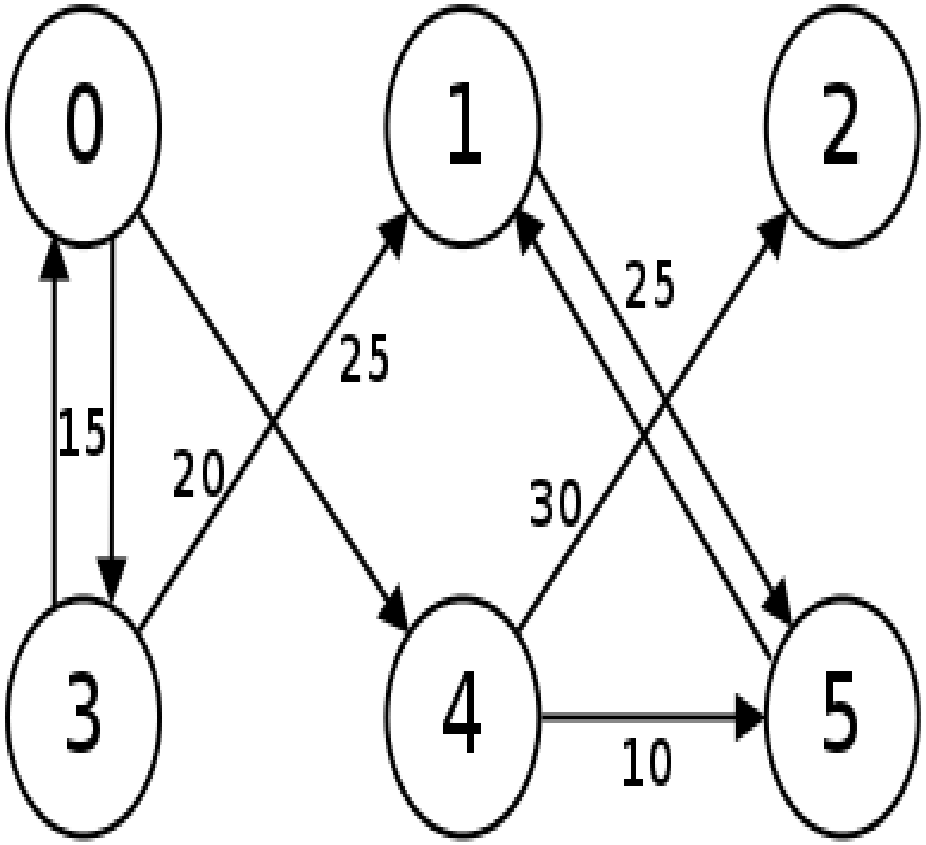
Adjacency Matrix

Exercise:
Write adjacency matrix for following graph



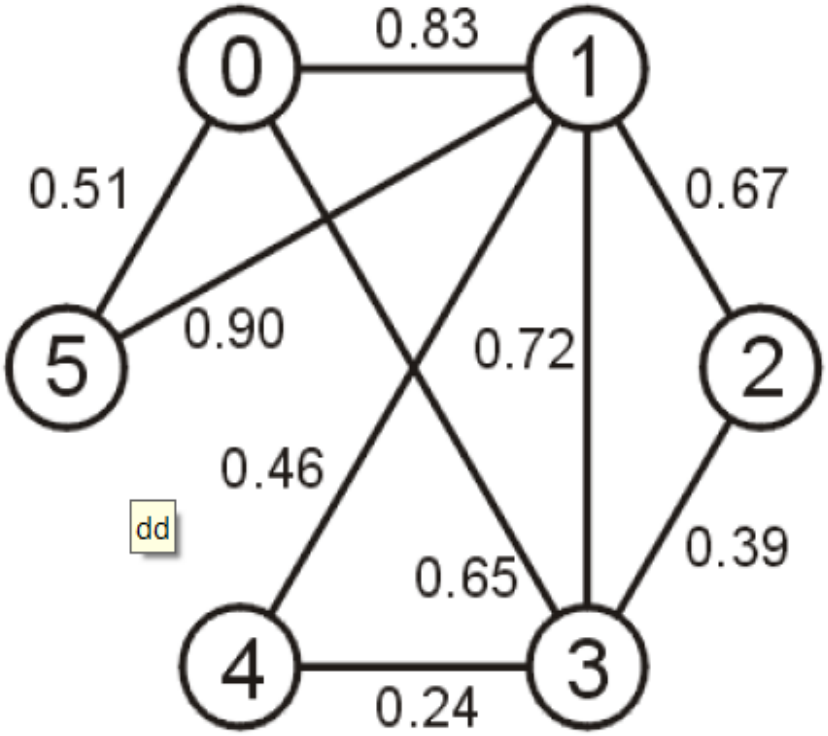
Adjacency Matrix

Answer



	0	1	2	3	4	5
0				15	20	
1						25
2						
3	15	25				
4			30			10
5		25				

Write the adjacency matrix for the following weighted graph



Incidence Matrix

Graph can be represented using a matrix of size total number of vertices by total number of edges.

Ex: A graph with 4 vertices and 6 edges can be represented using a matrix of 4X6 class.

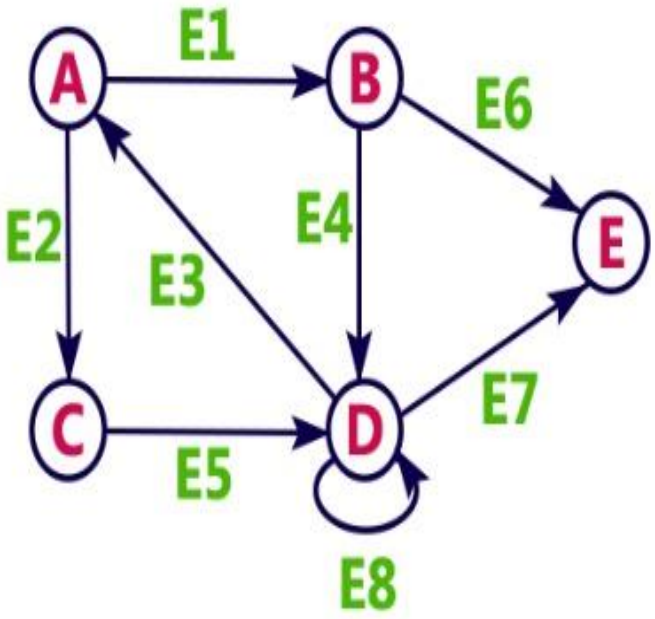
This matrix, rows represents vertices and columns represents edges.
Matrix is filled with either 0 or 1 or -1

0: represents row edge is not connected to column vertex.

1: represents row edge is connected as outgoing edge to column vertex

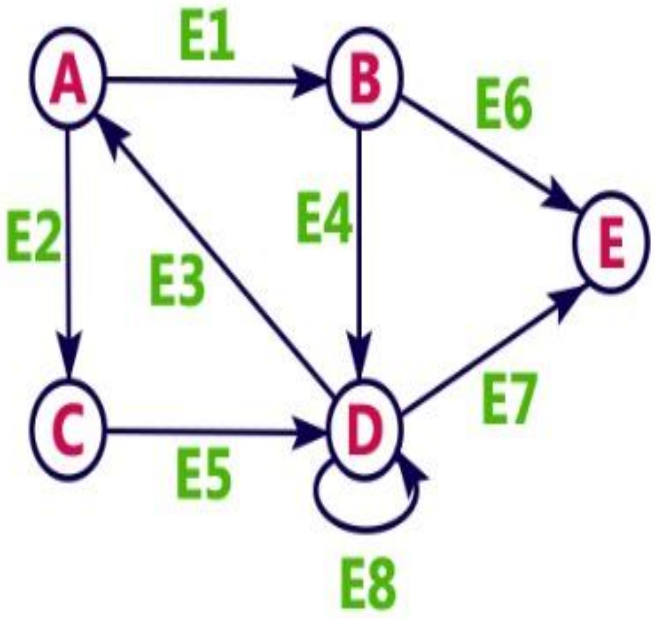
-1:represents row edge is connected as incoming edge to column vertex.

Incidence Matrix



	E1	E2	E3	E4	E5	E6	E7	E8
A	1	1	-1	0	0	0	0	0
B								
C								
D								
E								

Incidence Matrix

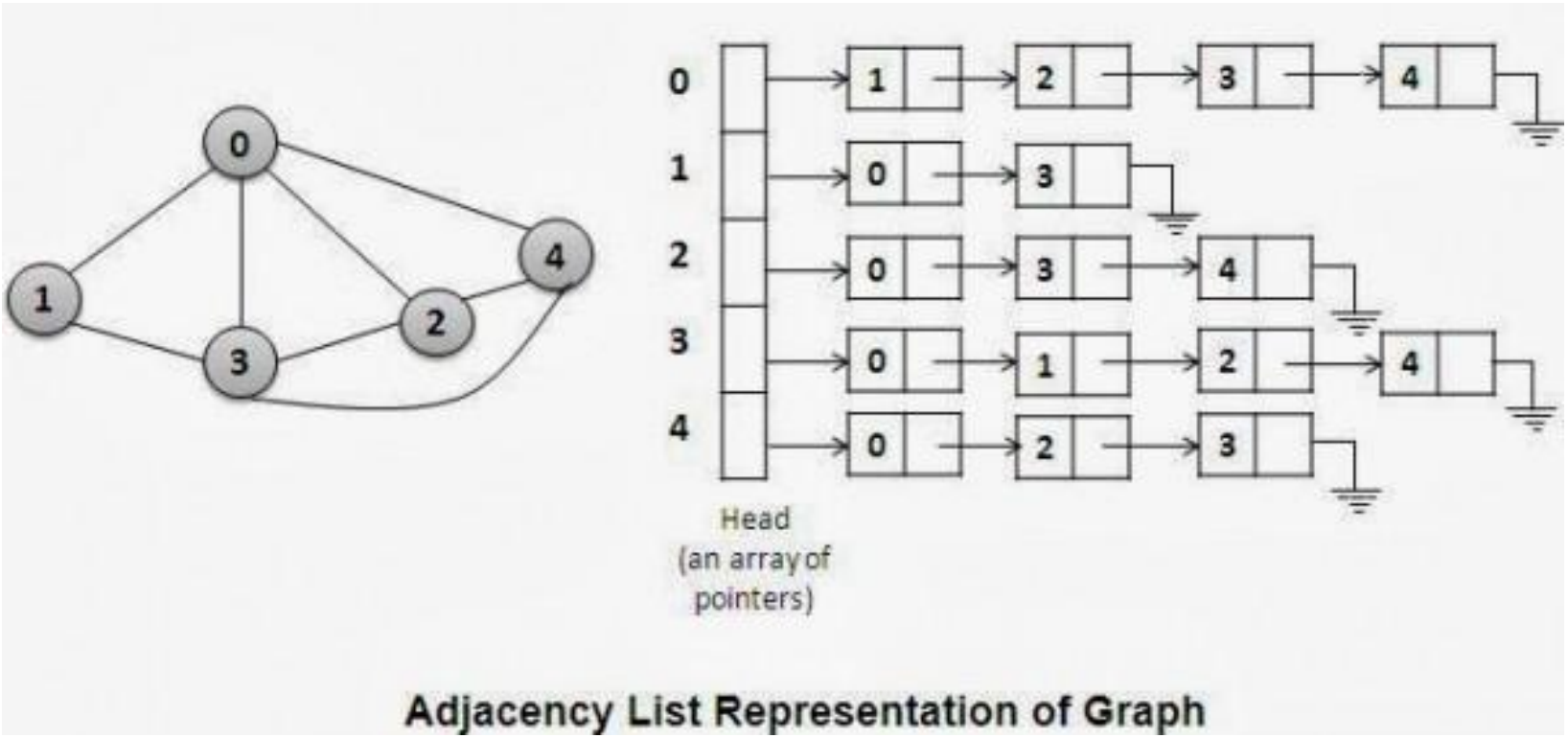


	E1	E2	E3	E4	E5	E6	E7	E8
A	1	1	-1	0	0	0	0	0
B	-1	0	0	1	0	1	0	0
C	0	-1	0	0	1	0	0	0
D	0	0	1	-1	-1	0	1	1
E	0	0	0	0	0	-1	-1	0

Adjacency List

- In this representation, every vertex of graph contains list of its adjacent vertices.

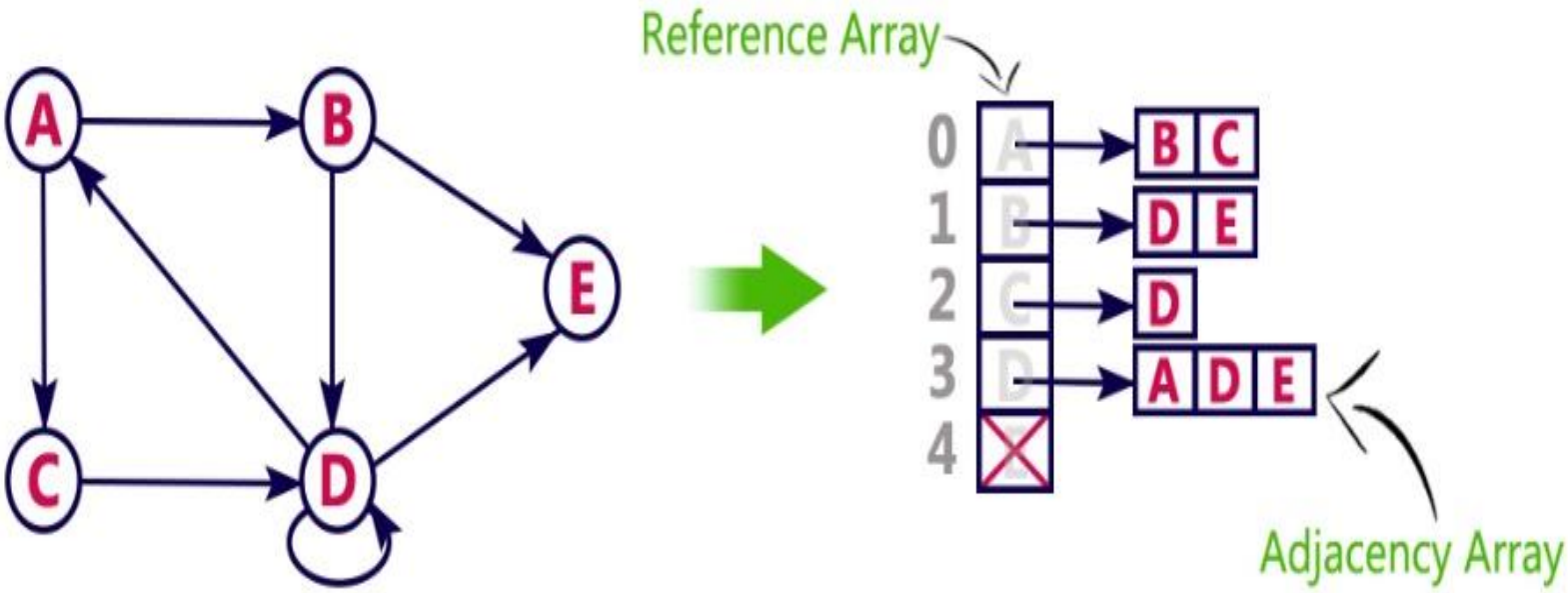
consider the following directed graph representation implemented using linked list.



Adjacency List Representation of Graph

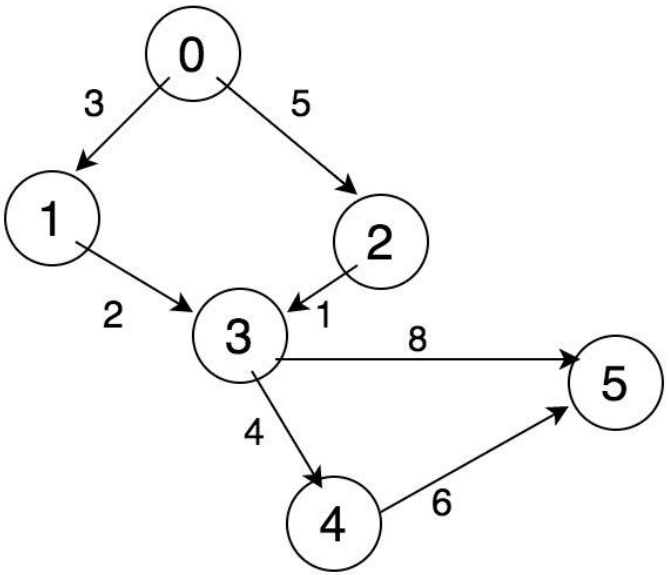
Adjacency List

This representation can also be implemented using array as follows..

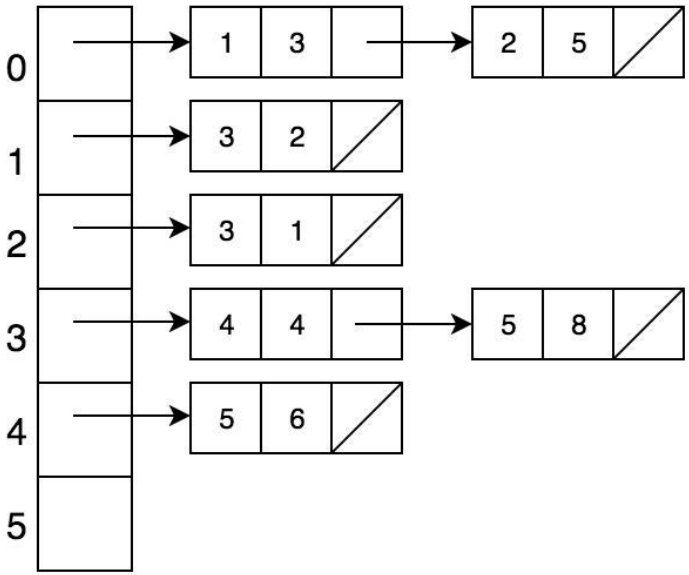


Adjacency list

Directed Graph



Adjacency List Representation



Exercise

Consider the graph G specified below.

$V(G) = \{1, 2, 3, 4, 5, 6\}$

$E(G) = \{(1,2), (2, 3), (3, 4), (5,1), (5, 6), (2, 6), (1, 6), (4, 6), (2, 4)\}$

Draw the graph G corresponding to the above specifications.

Obtain the following for the above graph:

(i) Adjacency matrix

(ii) Adjacency list

Graph Traversals

- Graph traversal is technique used for searching a vertex in a graph.
- The graph traversal is also used to decide the order of vertices to be visit in the search process.
- A graph traversal finds the edges to be used in the search process without creating loops that means using graph traversal we visit all vertices of graph without getting into looping path.

Graph Traversals

Two graph traversal techniques

- **DFS (Depth First Search)**
- **BFS (Breadth First Search)**

Depth First Search (DFS)

- DFS traversal of a graph, produces a **spanning tree** as final result. **Spanning Tree** is a graph without any loops.

- It uses **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal of a graph.

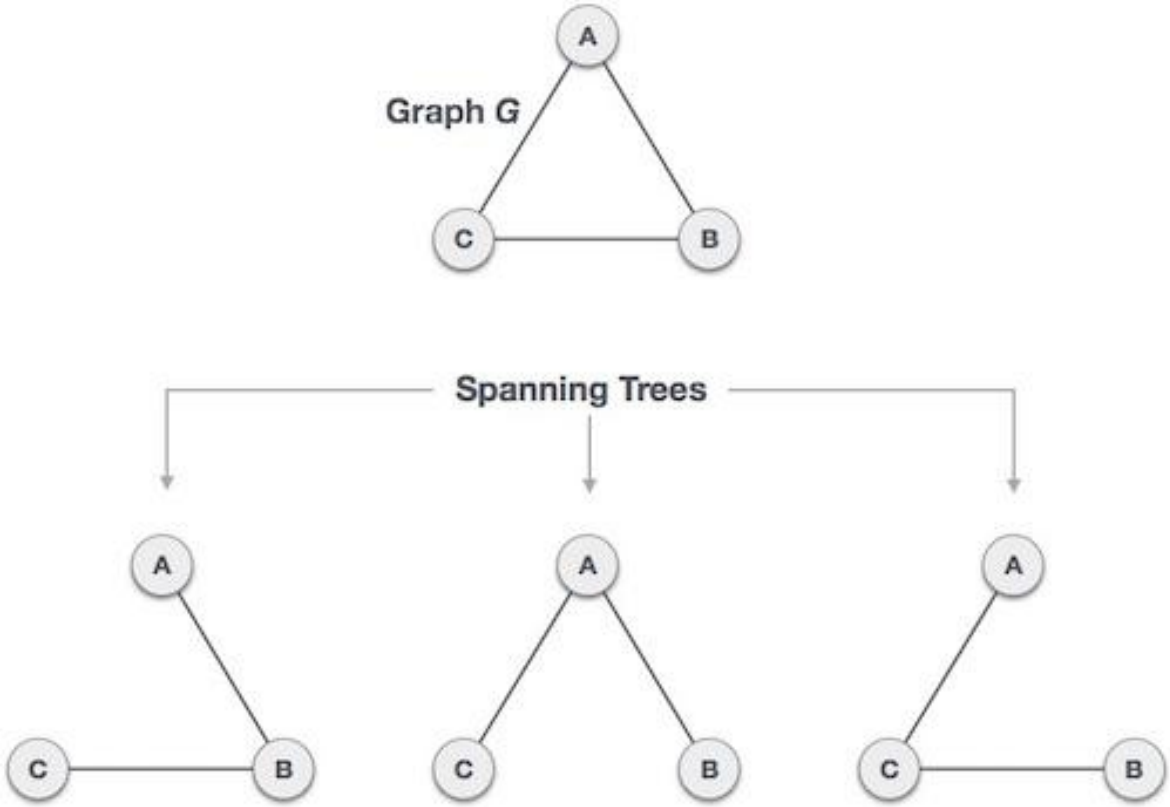
What is a spanning tree??

- A spanning tree is a **subset of Graph G**, which has all the vertices covered with minimum possible number of edges.

- Hence, a spanning tree does not have cycles and it cannot be disconnected.

- A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.

Spanning tree



- A complete undirected graph can have maximum $\mathbf{n^{n-2}}$ number of spanning trees, where **n** is the number of nodes.

- In the above addressed example, **n is 3,** hence $\mathbf{3^{3-2} = 3}$ spanning trees are possible.

General Properties of Spanning Tree

1. A connected graph G can have more than one spanning tree.

2. All possible spanning trees of graph G, have the same number of edges and vertices.

3. The spanning tree does not have any cycle (loops).

4. Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.

5. Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Spanning Tree Applications

- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis

Depth First Search (DFS)

Steps to implement DFS traversal

Step 1: Define a Stack of size total number of vertices in the graph.

Step 2: Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.

Step 3: Visit any one of the **adjacent** vertex of the vertex which is at top of the stack which is not visited and push it on to the stack.

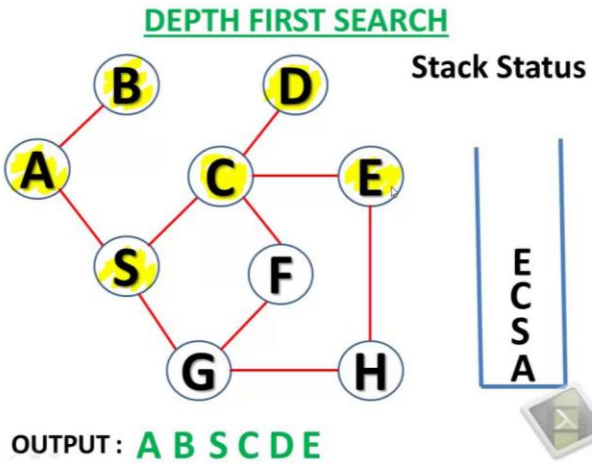
Depth First Search (DFS)

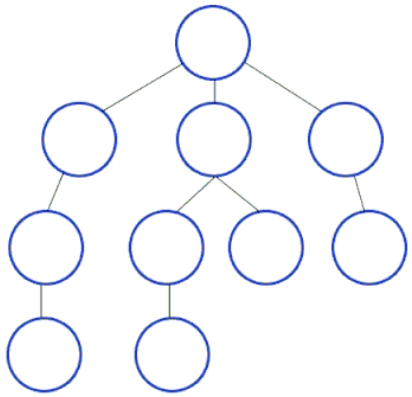
Step 4: Repeat step 3 until there are no new vertex to be visit from the vertex on top of the stack.

Step 5: When there is no new vertex to be visit then use **back tracking** and pop one vertex from the stack.

Step 6: Repeat steps 3, 4 and 5 until stack becomes Empty.

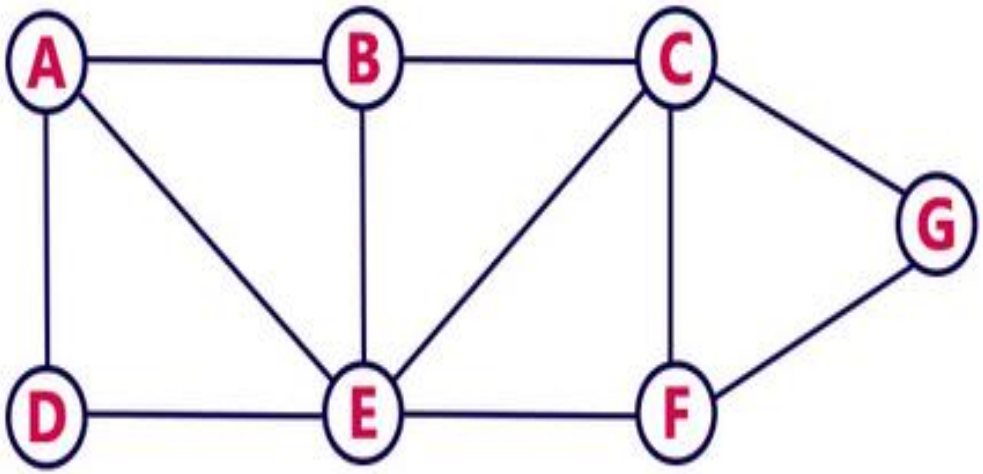
Step 7: When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph





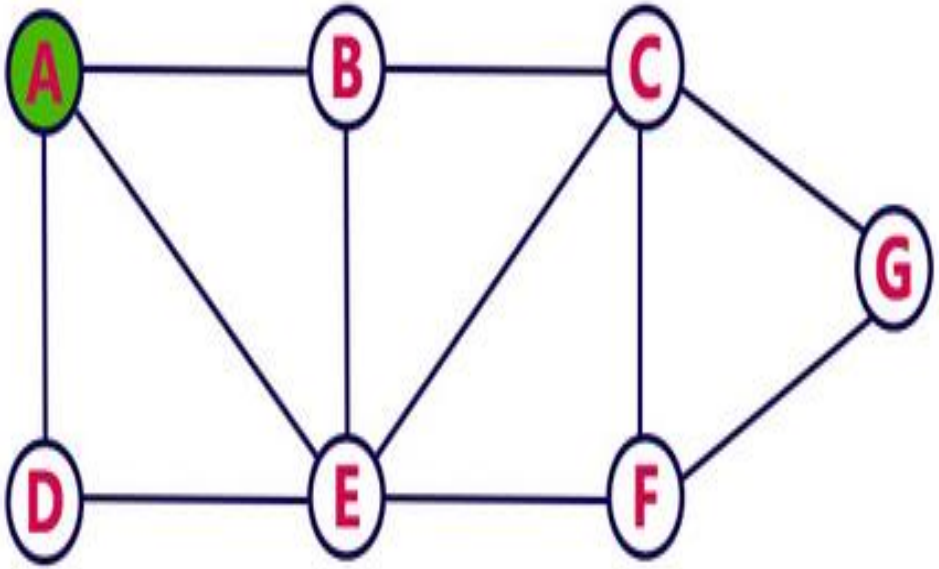
Depth First Search (DFS)

- Consider following example



Step 1:

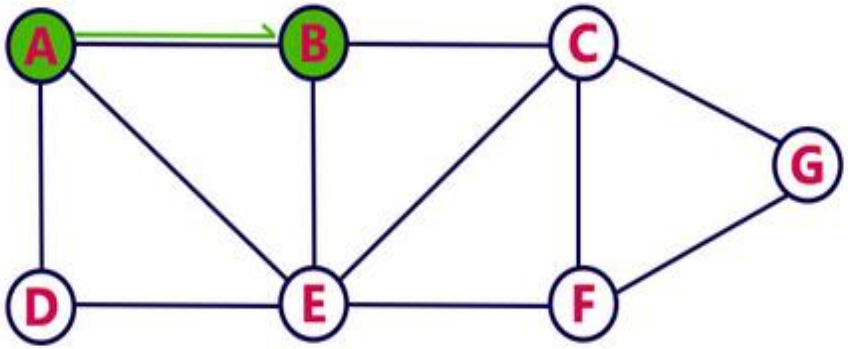
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.





Step 2:

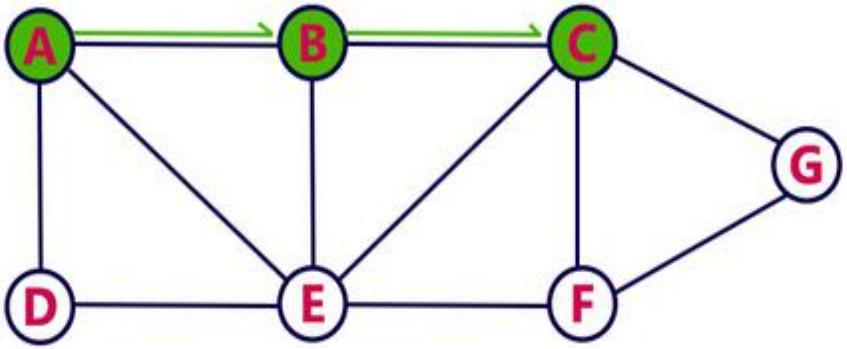
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.

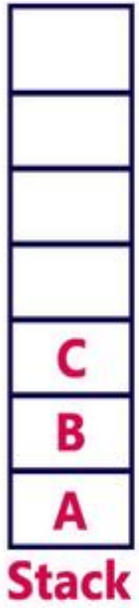




Step 3:

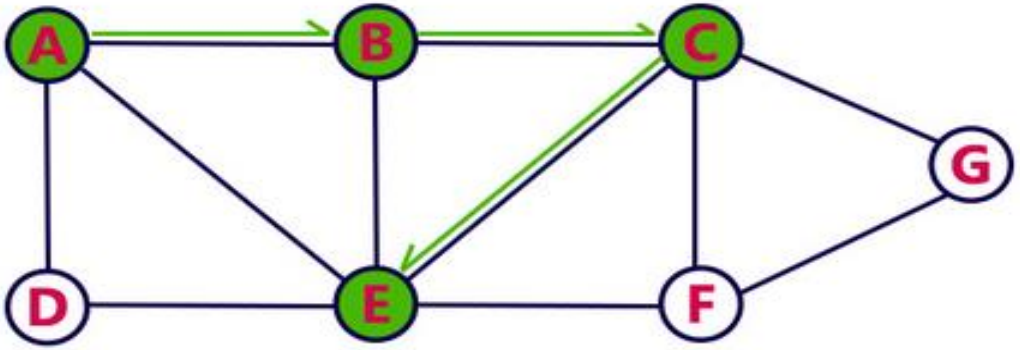
- Visit any adjacent vertext of **B** which is not visited (**C**).
- Push C on to the Stack.

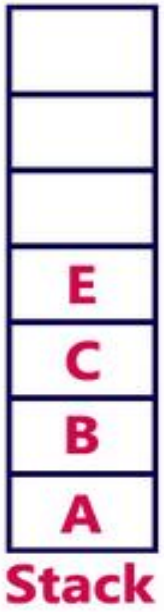




Step 4:

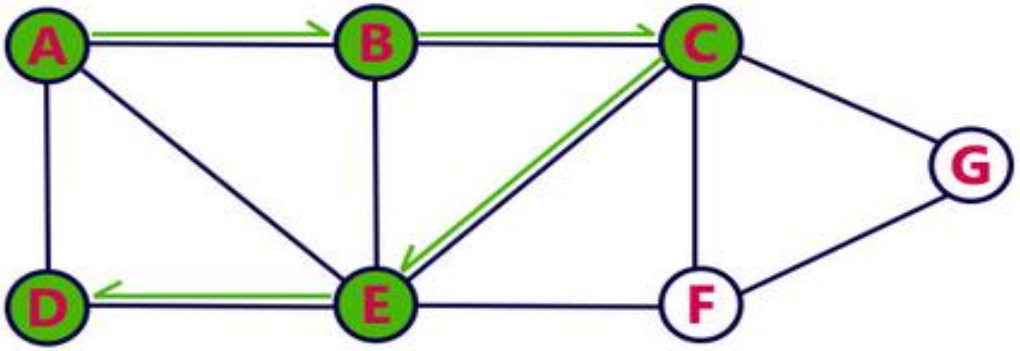
- Visit any adjacent vertext of **C** which is not visited (**E**).
- Push E on to the Stack





Step 5:

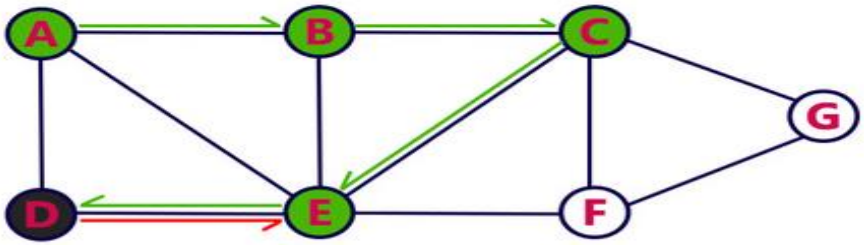
- Visit any adjacent vertext of **E** which is not visited (**D**).
- Push D on to the Stack





Step 6:

- There is no new vertiex to be visited from D. So use back track.
- Pop D from the Stack.



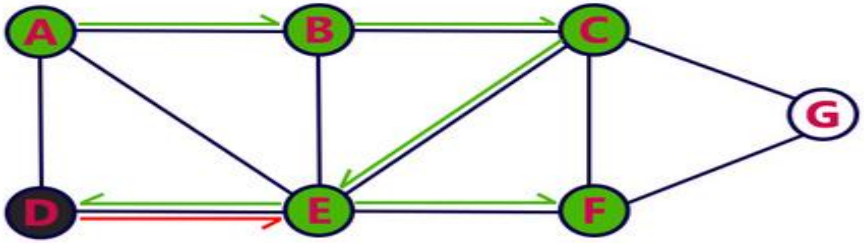
Stack

E
C
B
A

Stack

Step 7:

- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.

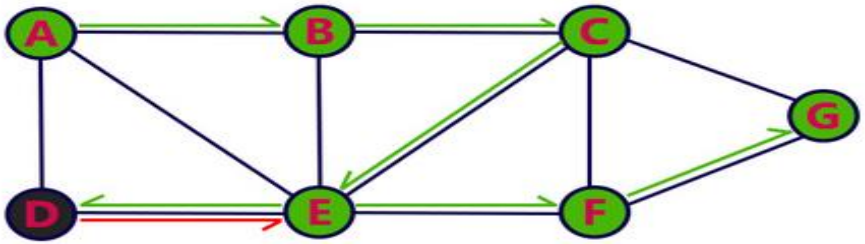


F
E
C
B
A

Stack

Step 8:

- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.

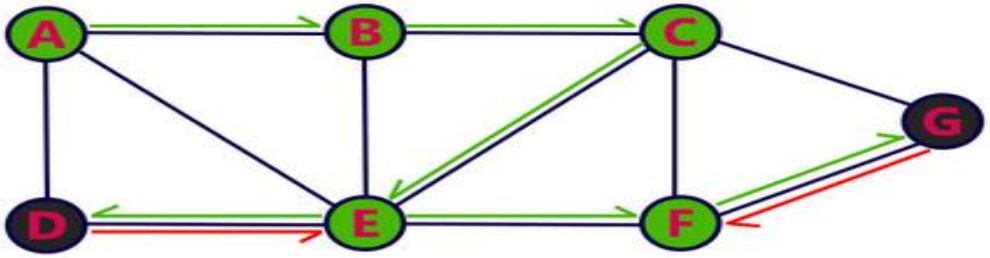


G
F
E
C
B
A

Stack

Step 9:

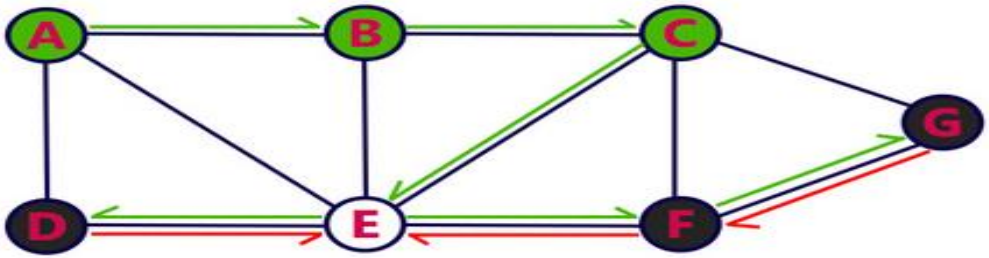
- There is no new vertiex to be visited from G. So use back track.
- Pop G from the Stack.

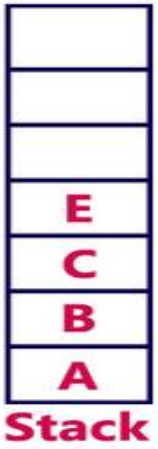




Step 10:

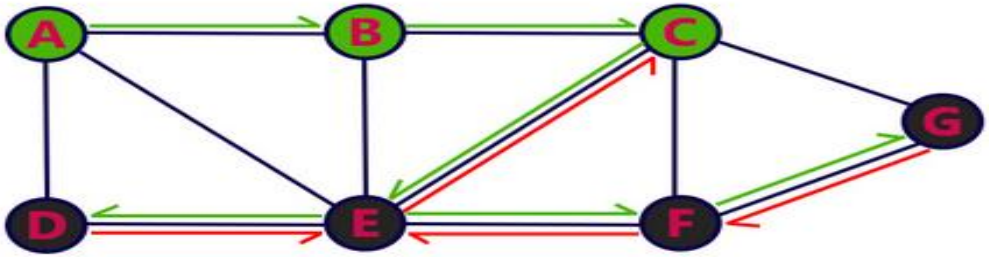
- There is no new vertiex to be visited from F. So use back track.
- Pop F from the Stack.

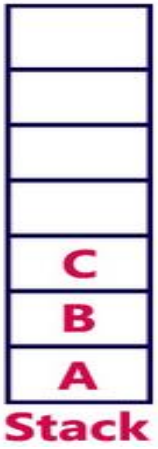




Step 11:

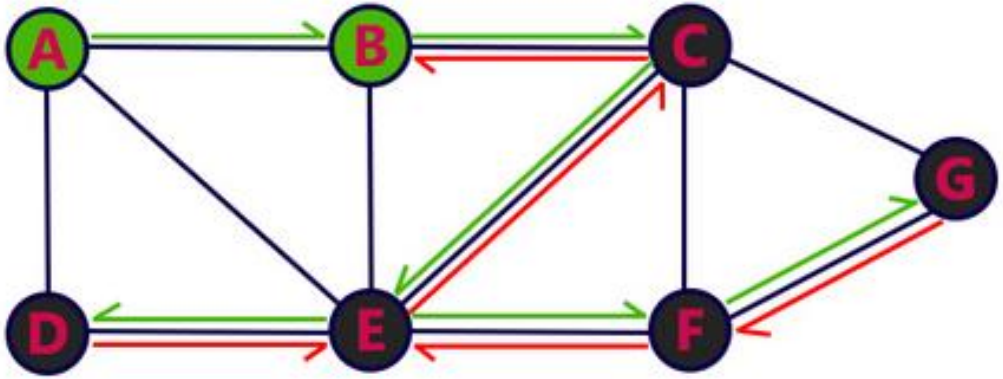
- There is no new vertiex to be visited from E. So use back track.
- Pop E from the Stack.





Step 12:

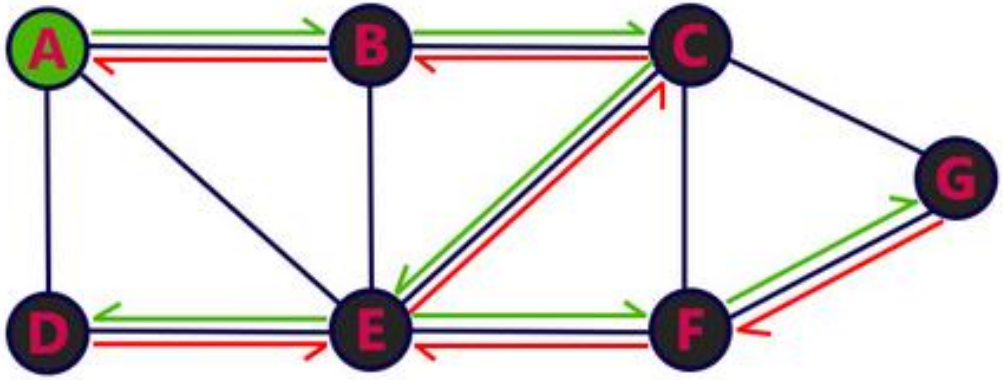
- There is no new vertiex to be visited from C. So use back track.
- Pop C from the Stack.

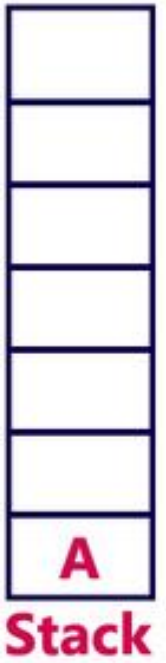




Step 13:

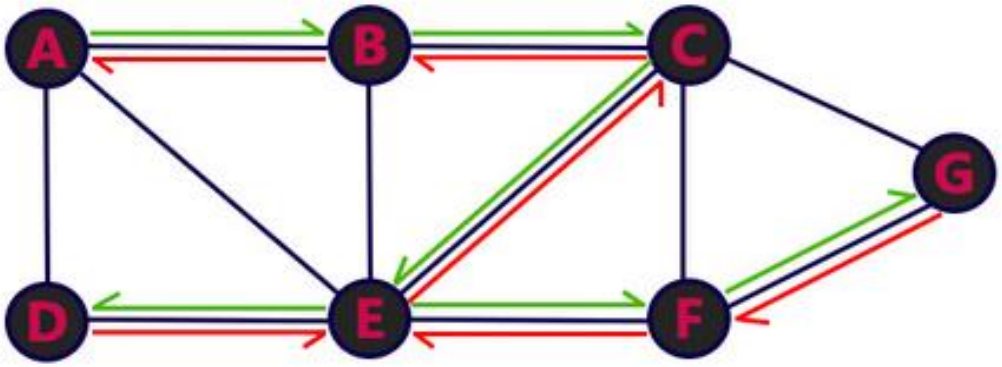
- There is no new vertiex to be visited from B. So use back track.
- Pop B from the Stack.





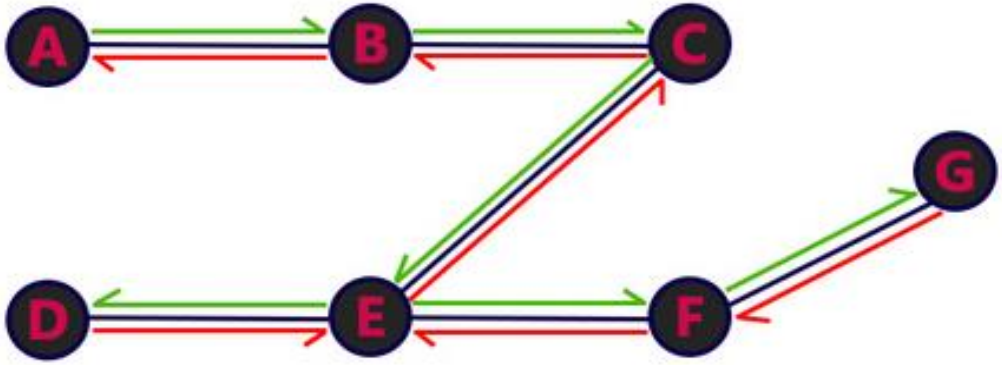
Step 14:

- There is no new vertiex to be visited from A. So use back track.
- Pop A from the Stack.



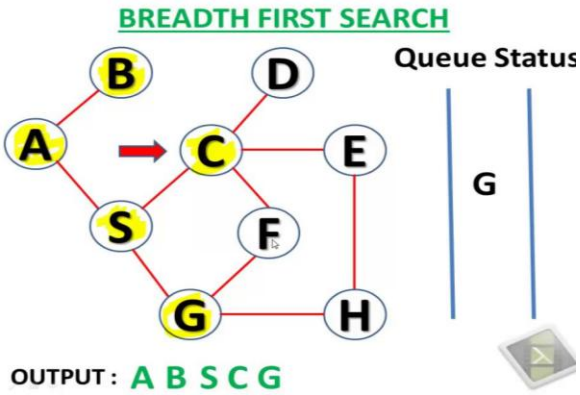


- Stack became Empty. So stop DFS Treversal.
- Final result of DFS traversal is following spanning tree.



BFS (Breadth First Search)

- BFS traversal of a graph, produces a **spanning tree** as final result.
- It uses **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal of a graph.

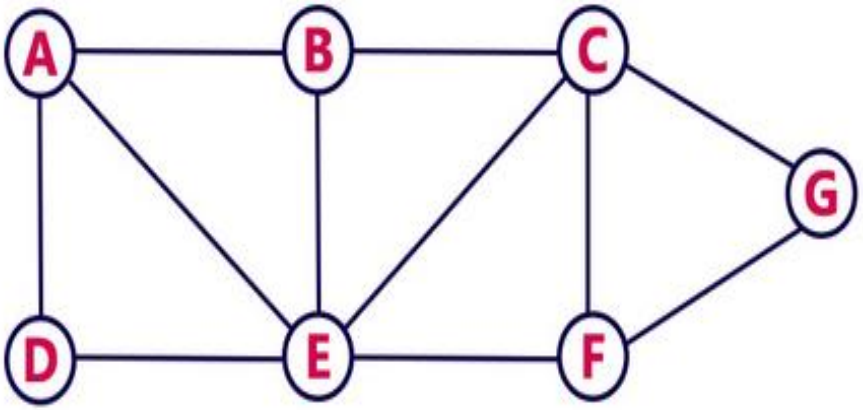


BFS (Breadth First Search)

Steps to implement BFS traversal...

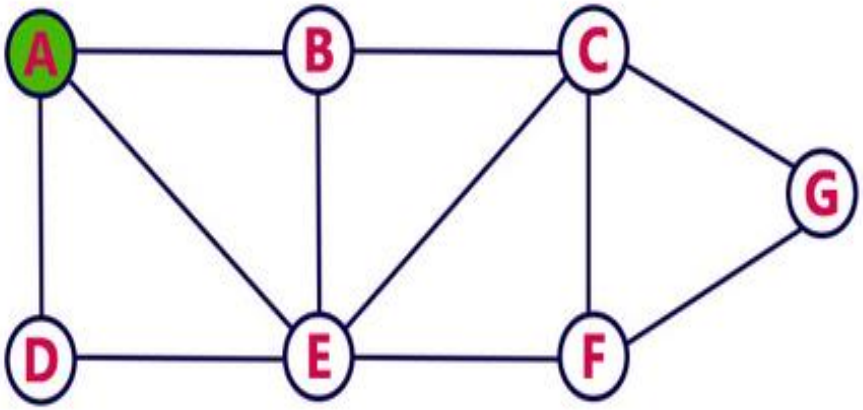
Step 1: Define a Queue of size total number of vertices in the graph.
Step 2: Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.
Step 3: Visit all the adjacent vertices of the vertex which is at front of the Queue which is not visited and insert them into the Queue.
Step 4: When there is no new vertex to be visit from the vertex at front of the Queue then delete that vertex from the Queue.
Step 5: Repeat step 3 and 4 until queue becomes empty.
Step 6: When queue becomes Empty, then produce final spanning tree by removing unused edges from the graph

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

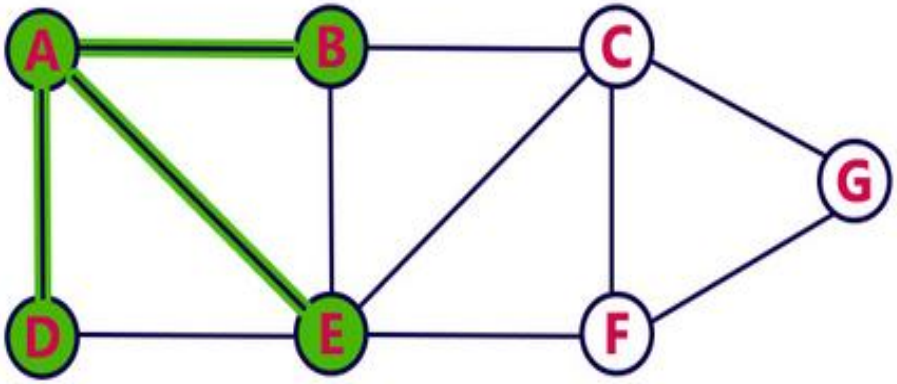


Queue



Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

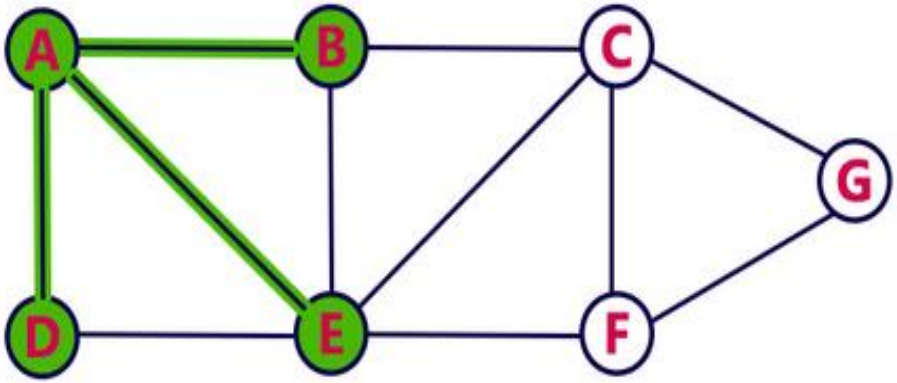


Queue

	D	E	B			

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

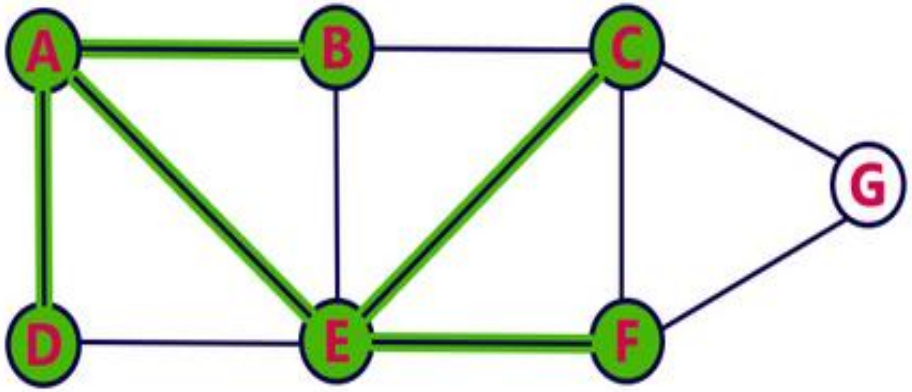


Queue

		E	B			

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

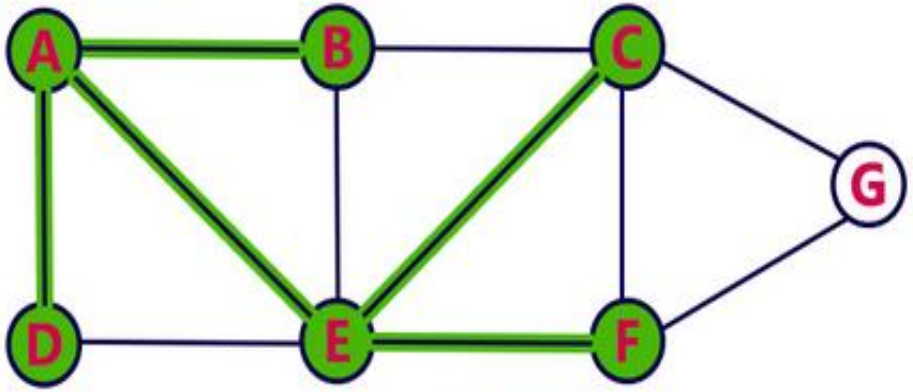


Queue

			B	C	F	

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

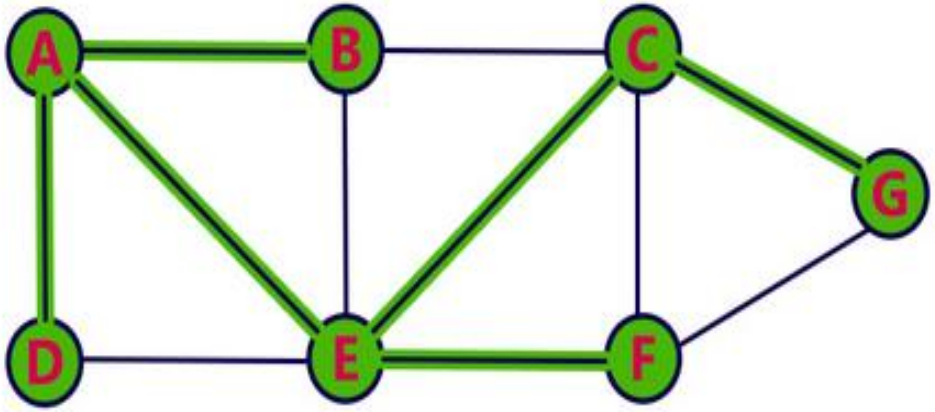


Queue

				C	F	

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

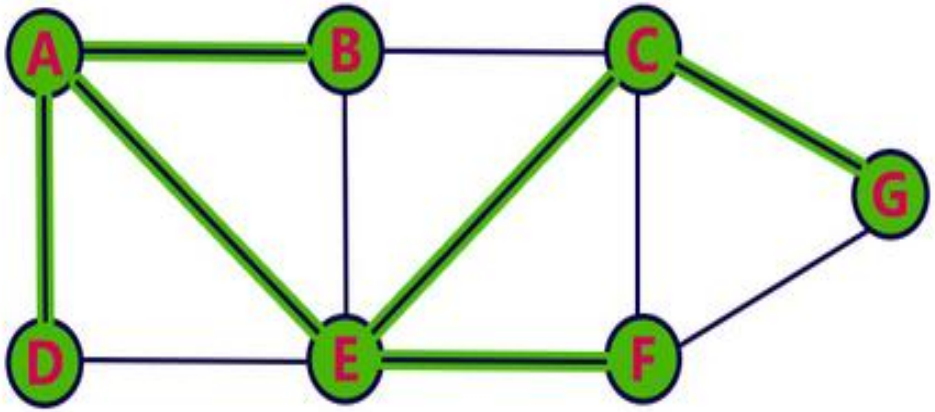


Queue

					F	G

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

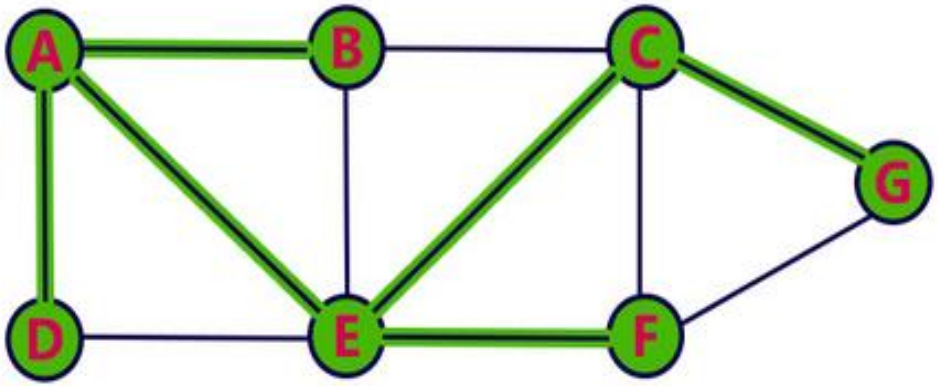


Queue

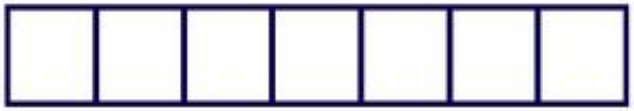
						G

Step 8:

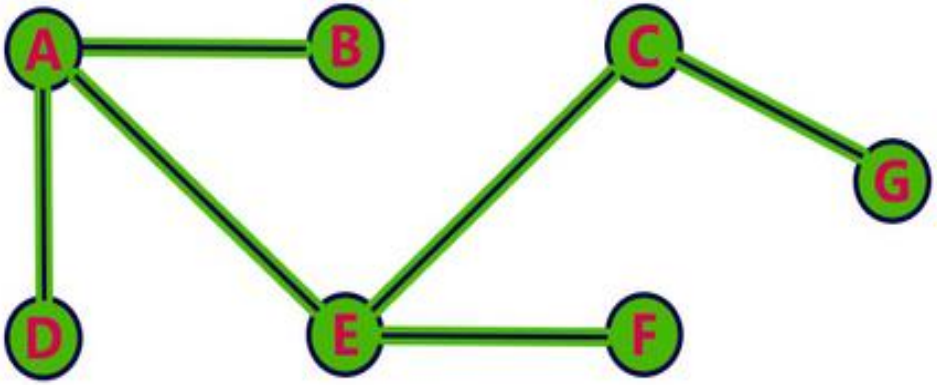
- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue



- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Difference between BFS and DFS

Breadth First Search (BFS)	Depth First Search (DFS)
BFS visit nodes level by level in Graph.	DFS visit nodes of graph depth wise. It visits nodes until reach a leaf or a node which doesn't have non-visited nodes.
A node is fully explored before any other can begin.	Exploration of a node is suspended as soon as another unexplored is found.
Uses Queue data structure to store Un-explored nodes.	Uses Stack data structure to store Un-explored nodes.
BFS is slower and require more memory.	DFS is faster and require less memory.

END